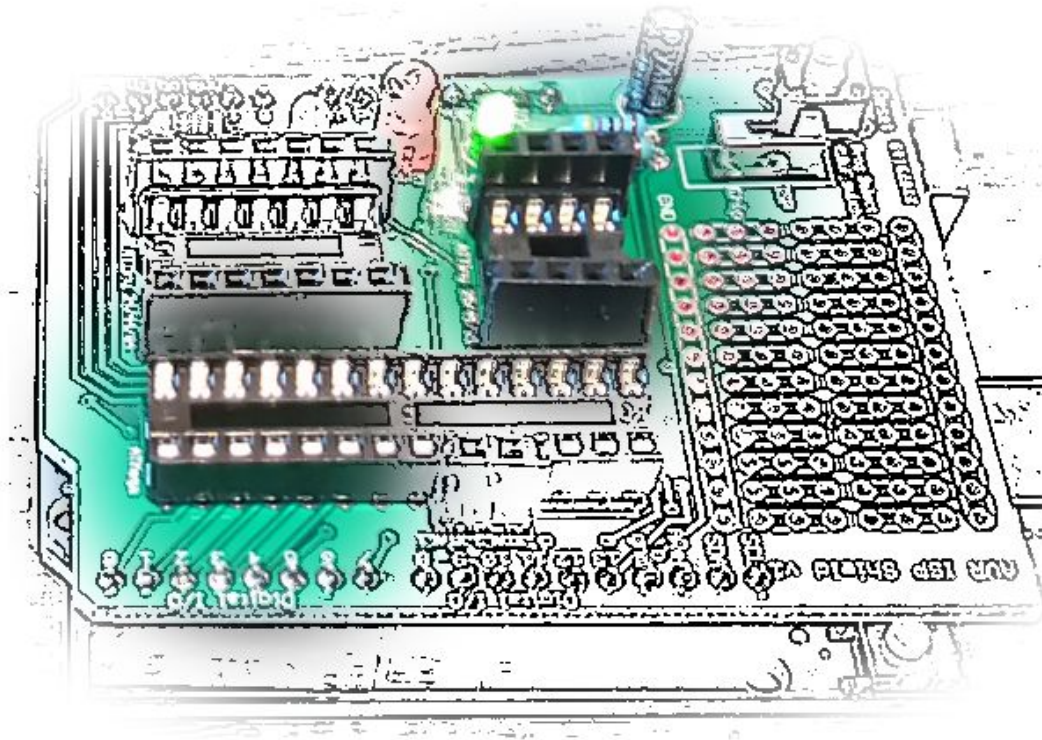


# AVR Assembler by Example



by Anthony Peck

## Quick Bio

Anthony graduated with a double major in Chemistry and Biochemistry from the University of Melbourne followed by Honours in Synthetic Organic Chemistry in the mid 1980s. He worked for a Danish company in industry for three years followed by four years at the Federal Department of Science (Patent Law) in Canberra.

He completed a postgraduate Diploma in Education and commenced his teaching career in 1992 and has taught STEM based subjects in the ACT, New South Wales, Western Australia, England and now Tasmania.

Anthony loves Chess, Motorcycling, Furry and Feathered friends, as well as dining with good mates. Married with two adult children, Anthony is a self-professed nerd and is still learning.

Since the late seventies Anthony has been programming in BASIC, PASCAL, C, Motorola 68000 Assembler, JAVA, Python, Ruby and now a smattering of AVR Assembler.

Personal Blog: <http://tertiusactus.blogspot.com/>



# AVR Assembler by example

## Introduction

What prompts a seemingly “normal” person to attempt to program a tiny microcontroller in assembler? There is the obvious intellectual challenge and stimulation, but also there is some merit in squeezing out the best of performances from these marvellous little devices.

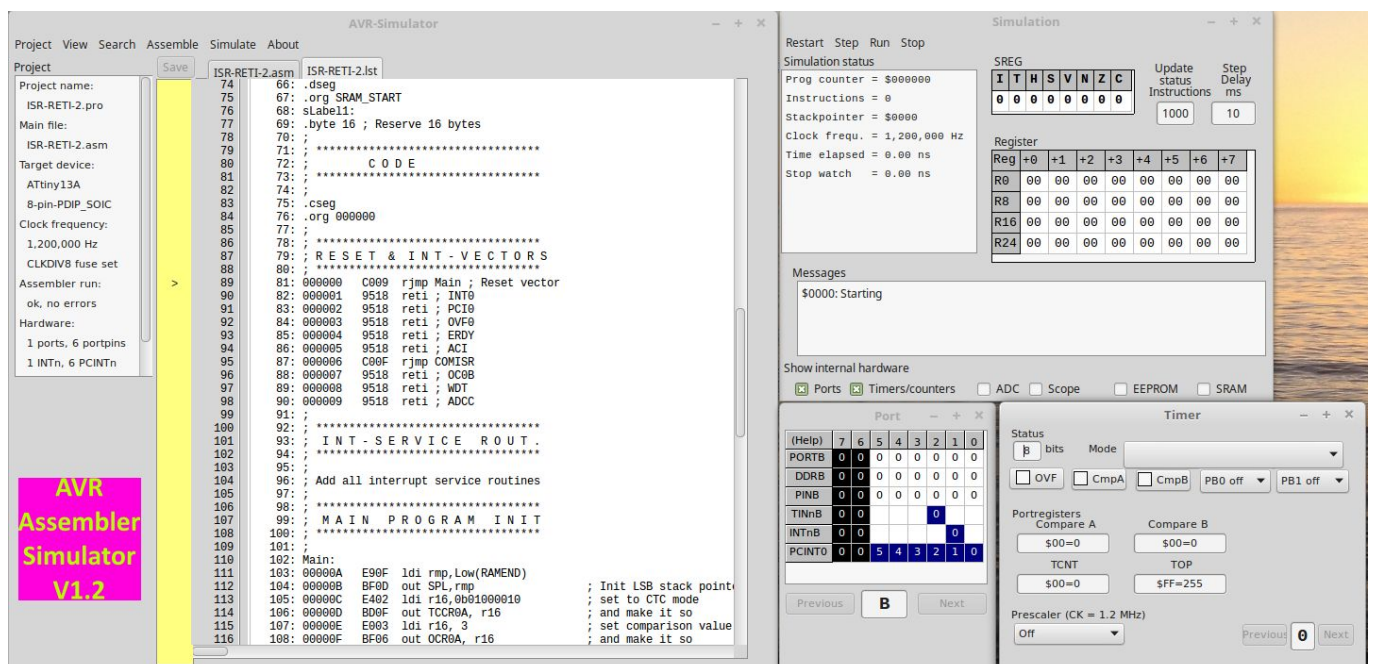
Programming an Amiga 500 in the eighties led eventually to using Motorola 68000 assembler to “shrinkify” firstly some DOS commands that seemed a bit bloaty, and then ultimately to a heap of utilities that ran very small, faster and more efficiently. As the PC evolved and commonly available machines had increasing amounts of ram, faster processors and bigger hard drives, the desire for programs that compiled to 120 bytes instead of 4 kbytes seemed less relevant.

Then along comes the AVR based Arduino revolution (32 kbytes with the Atmega328), the Attiny85 (8 kbytes) and our favourite the Attiny13 (1 kbyte). So now once again there seems to be a legitimate need for small, fast efficient code, just to see what is achievable by this little wonder. So we reluctantly turn away from the safe and reliable Arduino IDE and it’s C-like syntax and towards the nuts and bolts of chip programming - AVR assembler.

This resource was developed solely for our own purposes, educational and diversionary, and is in no way a definitive guide to AVR programming or the Attiny13 chip.

There is a lot of advice and resources available on the internet when starting out in this domain, and it would be a lengthy document were we to list all the influences, but it is important to point out some stellar contributions that have lit our way and which we have documented below.

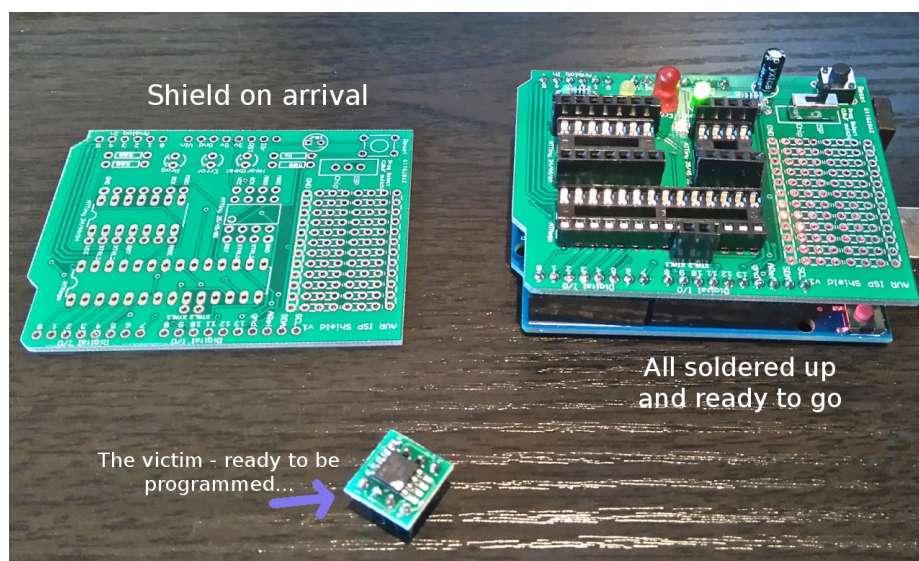
We do recommend the [textbook](#) “Some Assembly Required: Assembly Language Programming with the AVR Microcontroller” by Timothy S Margush. Also the [website](#) by Gerhard Schmidt is a tremendous resource, and indeed all of the code here was run firstly on [Gerd's AVR simulator](#), a wonderful tool for understanding the inner workings of an AVR chip (up to version 1.3 as of time of writing).



Our approach, for our own learning, has been to just simply document each instruction listed in the [Attiny13 datasheet](#) (pages 160-161) and then try in a trivial way to make a few lines of workable code to demonstrate the instruction, firstly using [Gerd's AVR simulator](#) and then the chips themselves. Along the journey some small complete programs have emerged, and we do feel that we have gained a lot of understanding and appreciation of the possibilities of this path.

Finishing up after a few months of writing and head scratching, it is really tempting to go back and make the earlier code more sophisticated, but that would eliminate the joy of the journey and focus only on the destination. Some of the early code here is a bit weird in hindsight, but it documents the process of increasing knowledge. It's certainly true that the code herein is in no way the most efficient, handsome, fastest or internally consistent code possible - again, just benchmarks on the way.

We program the attiny13 chips using an [avr isp shield](#) made by the innovative and very talented [Jia Huang](#). The Arduino IDE sets the fuses (all 1.2 Mhz for these projects), and then we use avrdude for the burning of the resultant hex files.



We get quite a kick out of this stuff (especially when it works), but huge thanks to the interwebs for your advice and tutorials, especially:

- [Gerhard Schmidt](#)
- [Łukasz Podkalicki](#)
- [Great Scott](#)
- [Julian Illett](#)
- [Big Clive](#)
- [EEVblog](#)
- [Afrotechmods](#)
- [Ben Eater](#)

These are great contributions to the community, and specifically to us. Thank you to all for your work in this domain. A special thank you to Gerhard Schmidt and Łukasz Podkalicki for your advice and support.

Speaking of contributions - if you find this booklet useful, please consider a small donation to help with updates, improvements, additions and the next production by clicking the link provided below.





## Contents

|                                                                                          |                    |
|------------------------------------------------------------------------------------------|--------------------|
| <a href="#">Introduction</a>                                                             | <a href="#">2</a>  |
| <a href="#">Common code (contents)</a>                                                   | <a href="#">7</a>  |
| <a href="#">ADC - Add with Carry (contents)</a>                                          | <a href="#">8</a>  |
| <a href="#">ADD - Add without carry (contents)</a>                                       | <a href="#">9</a>  |
| <a href="#">ADIW - Add Immediate to Word (contents)</a>                                  | <a href="#">10</a> |
| <a href="#">AND - Logical AND (contents)</a>                                             | <a href="#">11</a> |
| <a href="#">ANDI - Logical AND with Immediate (contents)</a>                             | <a href="#">12</a> |
| <a href="#">ASR - Arithmetic Shift Right (contents)</a>                                  | <a href="#">13</a> |
| <a href="#">BCLR - Bit Clear in SREG (contents)</a>                                      | <a href="#">14</a> |
| <a href="#">BSET - Bit Set in SREG (contents)</a>                                        | <a href="#">14</a> |
| <a href="#">BLD - Bit Load from the T Flag in SREG to a Bit in Register. (contents)</a>  | <a href="#">15</a> |
| <a href="#">BST - Bit Store from Bit in Register to T Flag in SREG. (contents)</a>       | <a href="#">15</a> |
| <a href="#">BRBC/S - Branch if Bit in SREG is Cleared/Set (contents)</a>                 | <a href="#">16</a> |
| <a href="#">BREAK - Break (contents)</a>                                                 | <a href="#">19</a> |
| <a href="#">CBI - Clear Bit in I/O Register (contents)</a>                               | <a href="#">20</a> |
| <a href="#">SBI - Set Bit in I/O Register (contents)</a>                                 | <a href="#">20</a> |
| <a href="#">CBR - Clear Bits in Register (contents)</a>                                  | <a href="#">21</a> |
| <a href="#">SBR- Set Bits in Register (contents)</a>                                     | <a href="#">21</a> |
| <a href="#">COM - One's Complement (contents)</a>                                        | <a href="#">23</a> |
| <a href="#">CP - Compare (contents)</a>                                                  | <a href="#">24</a> |
| <a href="#">CPC - Compare with Carry (contents)</a>                                      | <a href="#">25</a> |
| <a href="#">CPI - Compare with Immediate (contents)</a>                                  | <a href="#">26</a> |
| <a href="#">CPSE - Compare Skip if Equal (contents)</a>                                  | <a href="#">27</a> |
| <a href="#">DEC - Decrement (contents)</a>                                               | <a href="#">28</a> |
| <a href="#">EOR - Exclusive OR (contents)</a>                                            | <a href="#">29</a> |
| <a href="#">ICALL - Indirect Call to Subroutine (contents)</a>                           | <a href="#">30</a> |
| <a href="#">IJMP - Indirect Jump (contents)</a>                                          | <a href="#">31</a> |
| <a href="#">IN - Load an I/O Location to Register (contents)</a>                         | <a href="#">32</a> |
| <a href="#">OUT - Store Register to I/O Location (contents)</a>                          | <a href="#">32</a> |
| <a href="#">INC - Increment (contents)</a>                                               | <a href="#">33</a> |
| <a href="#">LD - Load Indirect from data space to Register using Index X (contents)</a>  | <a href="#">34</a> |
| <a href="#">ST - Store Indirect From Register to data space using Index X (contents)</a> | <a href="#">34</a> |
| <a href="#">LDS - Load Direct from data space (Y or Z) (contents)</a>                    | <a href="#">35</a> |
| <a href="#">STS - Store Direct to data space (Y or Z) (contents)</a>                     | <a href="#">35</a> |
| <a href="#">LPM - Load Program Memory (contents)</a>                                     | <a href="#">36</a> |
| <a href="#">LSL - Logical Shift Left (contents)</a>                                      | <a href="#">37</a> |
| <a href="#">LSR - Logical Shift Right (contents)</a>                                     | <a href="#">37</a> |
| <a href="#">MOV - Copy Register (contents)</a>                                           | <a href="#">38</a> |
| <a href="#">MOVW - Copy Register Word (contents)</a>                                     | <a href="#">39</a> |
| <a href="#">NEG - Two's Complement (contents)</a>                                        | <a href="#">40</a> |
| <a href="#">NOP - No Operation (contents)</a>                                            | <a href="#">41</a> |
| <a href="#">OR - Logical OR (contents)</a>                                               | <a href="#">41</a> |
| <a href="#">ORI - Logical OR with Immediate (contents)</a>                               | <a href="#">42</a> |
| <a href="#">OUT - Store Register to I/O Location (contents)</a>                          | <a href="#">43</a> |
| <a href="#">PUSH - Push Register on Stack (contents)</a>                                 | <a href="#">44</a> |
| <a href="#">POP - Pop Register from Stack (contents)</a>                                 | <a href="#">44</a> |

|                                                                                            |                           |
|--------------------------------------------------------------------------------------------|---------------------------|
| <a href="#"><u>RCALL - Relative Call to Subroutine (contents)</u></a>                      | <a href="#"><u>45</u></a> |
| <a href="#"><u>RET - Return from Subroutine (contents)</u></a>                             | <a href="#"><u>46</u></a> |
| <a href="#"><u>RETI - Return from Interrupt (contents)</u></a>                             | <a href="#"><u>46</u></a> |
| <a href="#"><u>Interval – making a real program (contents)</u></a>                         | <a href="#"><u>46</u></a> |
| <a href="#"><u>RJMP - Relative Jump (contents)</u></a>                                     | <a href="#"><u>51</u></a> |
| <a href="#"><u>ROL - Rotate Left through Carry (contents)</u></a>                          | <a href="#"><u>52</u></a> |
| <a href="#"><u>ROR - Rotate Right through Carry (contents)</u></a>                         | <a href="#"><u>53</u></a> |
| <a href="#"><u>Multiplying binary Numbers (contents)</u></a>                               | <a href="#"><u>54</u></a> |
| <a href="#"><u>Dividing binary Numbers (contents)</u></a>                                  | <a href="#"><u>55</u></a> |
| <a href="#"><u>SUB - Subtract without Carry (contents)</u></a>                             | <a href="#"><u>56</u></a> |
| <a href="#"><u>SBC - Subtract with Carry (contents)</u></a>                                | <a href="#"><u>56</u></a> |
| <a href="#"><u>SUBI - Subtract without Carry (contents)</u></a>                            | <a href="#"><u>57</u></a> |
| <a href="#"><u>SBCI - Subtract with Carry (contents)</u></a>                               | <a href="#"><u>57</u></a> |
| <a href="#"><u>SLEEP - Sleep mode (contents)</u></a>                                       | <a href="#"><u>57</u></a> |
| <a href="#"><u>SPM - Store Program Memory (contents)</u></a>                               | <a href="#"><u>58</u></a> |
| <a href="#"><u>SWAP - Swap Nibbles (contents)</u></a>                                      | <a href="#"><u>59</u></a> |
| <a href="#"><u>TST - Test for Zero or Minus (contents)</u></a>                             | <a href="#"><u>60</u></a> |
| <a href="#"><u>WDR - Watchdog Reset (contents)</u></a>                                     | <a href="#"><u>61</u></a> |
| <a href="#"><u>Some code which might be useful (contents)</u></a>                          | <a href="#"><u>66</u></a> |
| <a href="#"><u>PWM fade</u></a>                                                            | <a href="#"><u>66</u></a> |
| <a href="#"><u>Pseudo-random LED (8 bit lfsr) (contents)</u></a>                           | <a href="#"><u>72</u></a> |
| <a href="#"><u>Button Debouncing, input and output (contents)</u></a>                      | <a href="#"><u>77</u></a> |
| <a href="#"><u>Reading the ADC and comparing the result with a constant (contents)</u></a> | <a href="#"><u>83</u></a> |

## Common code ([contents](#))

All examples start with the following code from [http://www.avr-asm-tutorial.net/avr\\_sim/index\\_en.html](http://www.avr-asm-tutorial.net/avr_sim/index_en.html)

Explanations of what this code does have been saved for complete programs which appear later in the document. For now, it's best just to think of it as what needs to be defined before you get to the main code.

```
;
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 /-----| 8
; RESET o--|      |--o VCC
;      2 |      | 7
; PB3 o--|      |--o PB2
;      3 |      | 6
; PB4 o--|      |--o PB1
;      4 |      | 5
; GND o--|      |--o PB0
;      |-----|
;
;
; *****
; *
; *
; *****
;
.nolist
.include "tn13adef.inc" ; Define device
ATtiny13A
.list
;
; *****
; H A R D W A R E
; *****
;
; *****
; P O R T S   A N D   P I N S
; *****
;
.equ pOut = PORTB ; Define a output port
.equ pDir = DDRB ; Define a direction port
.equ pIn = PINB ; Define an input port
.equ bMyPinO = PORTB0 ; Define an output pin
.equ bDir0 = DDB0 ; Define a direction pin
.equ bMyIn = PINB0 ; Define an input pin
;
; *****
; A D J U S T A B L E   C O N S T
; *****
;
.equ clock=1200000 ; Define clock frequency
;
; *****
; F I X   &   D E R I V .   C O N S T
; *****
;
.equ cTc0Clk = clock / 256 ; Define from
clock
;

; *****
; R E G I S T E R S
; *****
;
; free: R0 to R14
.def rSreg = R15 ; Save/Restore status port
.def rmp = R16 ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...
;
; *****
; S R A M
; *****
;
; .dseg
; .org SRAM_START
sLabel1:
; .byte 16 ; Reserve 16 bytes
;
; *****
; C O D E
; *****
;
; .cseg
; .org 000000
;
; *****
; R E S E T   &   I N T - V E C T O R S
; *****
;
; rjmp Main ; Reset vector
; reti ; INTO
; reti ; PCIO
; reti ; OVFO
; reti ; ERDY
; reti ; ACI
; reti ; OC0A
; reti ; OC0B
; reti ; WDT
; reti ; ADCC
;
; *****
; I N T - S E R V I C E   R O U T .
; *****
;
; Add all interrupt service routines
```

## ADC - Add with Carry ([contents](#))

### Description:

Adds two registers and the contents of the C flag and places the result in the destination register Rd.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp           ; Init LSB stack pointer  
    .equ fred = 150        ; define fred (0x96)  
    .equ barney = 200      ; define barney (0xC8)  
    sei                   ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

Loop:

```
    ldi r16, fred          ; load 150 into register (0x96)  
    ldi r17, barney        ; load 200 into register (0xC8)  
    adc r16, r17           ; carry flag 01, r16 = 0x5E, 0x015E = 350  
    brcs Overflow         ; branch if carry set  
    rjmp Loop
```

Overflow:

```
    ldi r19, 0x01         ; carry overflow to r19 (0x01)  
    mov r20, r16          ; and lower byte to r20 (0x5E)  
    clc                   ; clear carry flag  
    clr r16               ; clear each used register  
    clr r17  
    clr r19  
    clr r20  
    rjmp Loop
```

```
; End of source code
```

## ADD - Add without carry ([contents](#))

### Description:

Adds two registers without the C flag and places the result in the destination register Rd.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

#### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp           ; Init LSB stack pointer  
    .equ fred = 64        ; define fred (0x40)  
    .equ barney = 32      ; define barney (0x20)  
    sei                   ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

#### Loop:

```
    clr r16  
    ldi r16, fred         ; load fred  
    ldi r17, barney       ; load barney  
    add r16, r17          ; add barney to fred, store as fred (0x60 = 96)  
    add r17, r17          ; double barney, store as barney (0x40 = 64)  
  
    clr r16               ; clear registers, start again  
    clr r17  
    rjmp Loop
```

```
; End of source code
```



## ADIW - Add Immediate to Word ([contents](#))

### Description:

Adds an immediate value (0-63) to a register pair and places the result in the register pair. This instruction operates on the upper four register pairs, and is well suited for operations on the pointer registers.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 758            ; define fred (0x02F6)  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    clr r26  
    clr r27  
    ldi r27, high(fred)        ; store high part of fred (0x02)  
    ldi r26, low(fred)         ; store low part of fred (0xF6)  
    adiw r27:r26, 15           ; add 15 to fred (0x0305 = 773)  
  
    rjmp Loop
```

```
; End of source code
```

## AND - Logical AND ([contents](#))

Description:

Performs the logical AND between the contents of register Rd and register Rr and places the result in the destination register Rd.

AND Truth Table

| Inputs |   | Output  |
|--------|---|---------|
| A      | B | Y = A.B |
| 0      | 0 | 0       |
| 0      | 1 | 0       |
| 1      | 0 | 0       |
| 1      | 1 | 1       |

e.g. 0x35 AND 0x3C

|        |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|
| Byte 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| Byte 2 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| AND    | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 0b00110101     ; define fred 0x35  
    .equ barney = 0b00111100   ; define barney 0x3C  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    clr r16                    ; clear used registers  
    clr r17  
    ldi r17, fred              ; load fred  
    ldi r16, barney            ; load barney  
    and r16, r17               ; 0b00110101 AND 0b00111100 = 0b00110100  
(0x34)
```

```
    rjmp Loop
```

```
; End of source code
```

## ANDI - Logical AND with Immediate ([contents](#))

### Description:

Performs the logical AND between the contents of register Rd and a constant and places the result in the destination register Rd.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 0b00110101     ; define fred (0x35)  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    clr r16  
    ldi r16, fred              ; load fred into r16  
    andi r16, 0b00111100      ; "and" immediate with 0x3C  
                                ; (result 0b00110100 = 0x34)  
    rjmp Loop
```

```
; End of source code
```

## ASR - Arithmetic Shift Right ([contents](#))

### Description:

Shifts all bits in Rd one place to the right. Bit 7 is held constant. Bit 0 is loaded into the C flag of the SREG. This operation effectively divides a signed value by two without changing its sign. The carry flag can be used to round the result.

```
; *****
;   M A I N   P R O G R A M   I N I T
; *****

Main:
    ldi rmp,Low(RAMEND)
    out SPL,rmp           ; Init LSB stack pointer
    .equ fred = 0b01010000 ; define fred (0x50)
    sei                   ; Enable interrupts

; *****
;   P R O G R A M   L O O P
; *****

Loop:
    clr r16
    ldi r16, fred
    asr r16                ; 0b00101000 (0x28)

    asr r16                ; 0b00010100 (0x14)

    asr r16                ; 0b00001010 (0x0A)

    rjmp Loop

; End of source code
```

|                                                                                                                 |                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>BCLR - Bit Clear in SREG (<a href="#">contents</a>)</b><br><br>Description:<br>Clears a single flag in SREG. | <b>BSET - Bit Set in SREG (<a href="#">contents</a>)</b><br><br>Description:<br>Sets a single flag or bit in SREG. |
|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|

```
; *****
;  M A I N    P R O G R A M    I N I T
;  *****
```

Main:

```
    ldi rmp,Low(RAMEND)
    out SPL,rmp      ; Init LSB stack pointer
    ldi r16, 0
    sei              ; Enable interrupts
```

```
; *****
;    P R O G R A M    L O O P
;  *****
```

Loop:

```
    bset 0          ; set C flag
    bclr 0          ; clear C flag
    bset 1          ; set Z flag
    bclr 1          ; clear Z flag
    bset 2          ; set N flag
    bclr 2          ; clear N flag
    bset 3          ; set V flag
    bclr 3          ; clear V flag
    bset 4          ; set S flag
    bclr 4          ; clear S flag
    bset 5          ; set H flag
    bclr 5          ; clear H flag
    bset 6          ; set T flag
    bclr 6          ; clear T flag
    bset 7          ; set I flag
    bclr 7          ; clear I flag
```

```
    rjmp Loop
```

```
; End of source code
```

**BLD - Bit Load from the T Flag in SREG to a Bit in Register. ([contents](#))****Description:**

Copies the T flag in the SREG (status register) to bit b in register Rd.

**BST - Bit Store from Bit in Register to T Flag in SREG. ([contents](#))****Description:**

Stores bit b from Rd to the T flag in SREG (status register).

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

**Main:**

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp          ; Init LSB stack pointer  
    sei                  ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

**Loop:**

```
    clr r16  
    bset 6                ; set T flag  
    bld r16, 1            ; load T flag into bit 1 of r16  
                           ; (0b000000010 = 0x02)  
    ldi r16, 0b00000000   ; T flag will be loaded as zero  
    bst r16, 2            ; store bit 2 from r16 in T flag  
  
    rjmp Loop
```

```
; End of source code
```



## BRBC/S - Branch if Bit in SREG is Cleared/Set ([contents](#))

### Description:

Conditional relative branch. Tests a single bit in SREG and branches relatively to PC if the bit is cleared/set. This instruction branches relatively to PC in either direction ( $PC - 63 \leq \text{destination} \leq PC + 64$ ). The parameter k is the offset from PC and is represented in two's complement form.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp                ; Init LSB stack pointer  
    .equ fred = 42              ; define fred  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    ldi r16, fred                ; load r16 with 42 (0x2A)  
    bset 1                       ; set the Z flag  
    brbc 1, Never                ; if Z cleared, branch to never  
    bclr 1                       ; clear the Z flag  
    brbc 1, Always               ; if Z cleared, branch to always
```

#### Never:

```
    ldi r16, 0                  ; zero never loaded
```

#### Always:

```
    ldi r16, 1                  ; one always loaded
```

```
    rjmp Loop
```

```
; End of source code
```

**BRBC/S - Branch if Bit in SREG is Cleared/Set** works for all SREG, so it is not necessary to learn or use all of these alias branch instructions:

### SREG – Status Register

| Bit           | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
|---------------|-----|-----|-----|-----|-----|-----|-----|-----|
| 0x3F          | I   | T   | H   | S   | V   | N   | Z   | C   |
| Read/Write    | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Initial Value | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |

|                |      |                                                                                            |
|----------------|------|--------------------------------------------------------------------------------------------|
| BRBS 0, branch | BRCS | Branch if Carry Set if (C = 1) then $PC \leftarrow PC + k + 1$                             |
| BRBC 0, branch | BRCC | Branch if Carry Cleared if (C = 0) then $PC \leftarrow PC + k + 1$                         |
| BRBS 1, branch | BREQ | Branch if Equal if (Z = 1) then $PC \leftarrow PC + k + 1$                                 |
| BRBC 1, branch | BRNE | Branch if Not Equal if (Z = 0) then $PC \leftarrow PC + k + 1$                             |
| BRBS 2, branch | BRMI | Branch if Minus if (N = 1) then $PC \leftarrow PC + k + 1$                                 |
| BRBC 2, branch | BRPL | Branch if Plus if (N = 0) then $PC \leftarrow PC + k + 1$                                  |
| BRBS 3, branch | BRVS | Branch if Overflow Flag is Set if (V = 1) then $PC \leftarrow PC + k + 1$                  |
| BRBC 3, branch | BRVC | Branch if Overflow Flag is Cleared if (V = 0) then $PC \leftarrow PC + k + 1$              |
| BRBS 4, branch | BRGE | Branch if Greater or Equal, Signed if ( $N \oplus V = 0$ ) then $PC \leftarrow PC + k + 1$ |
| BRBC 4, branch | BRLT | Branch if Less Than Zero, Signed if ( $N \oplus V = 1$ ) then $PC \leftarrow PC + k + 1$   |
| BRBS 5, branch | BRHS | Branch if Half Carry Flag Set if (H = 1) then $PC \leftarrow PC + k + 1$                   |
| BRBC 5, branch | BRHC | Branch if Half Carry Flag Cleared if (H = 0) then $PC \leftarrow PC + k + 1$               |
| BRBS 6, branch | BRTS | Branch if T Flag Set if (T = 1) then $PC \leftarrow PC + k + 1$                            |
| BRBC 6, branch | BRTC | Branch if T Flag Cleared if (T = 0) then $PC \leftarrow PC + k + 1$                        |
| BRBS 7, branch | BRIE | Branch if Interrupt Enabled if (I = 1) then $PC \leftarrow PC + k + 1$                     |
| BRBC 7, branch | BRID | Branch if Interrupt Disabled if (I = 0) then $PC \leftarrow PC + k + 1$                    |

Despite the multiple redundancy of names, it might be a good time to look at how BRBC/S or their aliases might be used in assembler programs.

## BRBC/S - Branch if Bit in SREG is Cleared/Set ([contents](#))

Aim: to code an assembly version of arduino blink, with 10 rapid blinks, a pause, and then repeat. Uses crude delay loops galloping along at 1.2MHz. Branches shown in bold. Note this is not great code (micro running at full speed and just counting), and we will do some alternatives later in the document.

```
; M A I N   P R O G R A M   I N I T
; *****

Main:
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer
    .equ blinks = 10          ; define number of blinks
    ldi r16, 0b00000001       ; load 'on'
    out DDRB, r16             ; and PB0 is output
    sei                       ; Enable interrupts

    .macro timer
; See http://www.bretmulvey.com/avrdelay.html (AVR at 1.2MHz)
; 200ms = 2, 56, 174 (passed to this macro when called)
; 1sec = 7, 23, 107 (passed to this macro when called)

        ldi r18, @0           ; first value loaded into register
        ldi r19, @1           ; second value loaded into register
        ldi r20, @2           ; third value loaded into register

L1:
        dec r20                ; r20 decrement by one
        brbc 1, L1           ; if not zero (Z=0), branch back to loop (L1)
        dec r19                ; r19 decrement by one
        brbc 1, L1           ; if not zero (Z=0), branch back to loop (L1)
        dec r18                ; r18 decrement by one
        brbc 1, L1           ; if not zero (Z=0), branch back to loop (L1)

    .endmacro

;   P R O G R A M   L O O P
; *****

Loop:
    ldi r17, blinks            ; load the number of blinks to r17

Blinkloop:                    ; loop label
    ldi r16, 0b00000001       ; PB0 wants to be on
    out PORTB, r16            ; and is now on
    timer 2,56,174            ; delay 200ms
    ldi r16, 0b00000000       ; PB0 wants to be off
    out PORTB, r16            ; and is now off
    timer 2,56,174            ; delay 200ms
    dec r17                    ; r17 decrement by one
    brbc 1, Blinkloop        ; if not zero, branch back to loop (Blinkloop)
    timer 7, 23, 107          ; delay 1 second
    rjmp Loop                  ; start again

; End of source code
```

## BREAK - Break ([contents](#))

### Description:

The BREAK instruction is used by the On-Chip Debug system, and is normally not used in the application software. When the BREAK instruction is executed, the AVR CPU is set in the Stopped Mode. This gives the On-Chip Debugger access to internal resources.

If any lock bits are set, or either the JTAGEN or OCDEN fuses are unprogrammed, the CPU will treat the BREAK instruction as a NOP and will not enter the Stopped Mode.

From the JTAG ICE manual:

*A software breakpoint is a break instruction placed in Flash memory. When this instruction is executed, it will break the program execution. When placing a breakpoint on an instruction in AVR Studio, this instruction is physically rewritten as a break instruction in the AVR Flash memory. When reaching this instruction the operation is halted. To continue execution a “start” command has to be given from the OCD logic. When starting the execution, the instruction replaced by software break instruction is executed before continuing to execute instructions from the Flash memory.*

| CBI - Clear Bit in I/O Register ( <a href="#">contents</a> )                                                                                 | SBI - Set Bit in I/O Register ( <a href="#">contents</a> )                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Description:<br/>Clears a specified bit in an I/O register. This instruction operates on the lower 32 I/O registers - addresses 0-31.</p> | <p>Description:<br/>Sets a specified bit in an I/O register. This instruction operates on the lower 32 I/O registers - addresses 0-31</p> |

```
; *****
;   M A I N   P R O G R A M   I N I T
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)
    out SPL,rmp           ; Init LSB stack pointer
    ldi r16, 0b00001111   ; PB0-PB3 set to output
    out DDRB, r16         ; make it so in DDRB
    sei                   ; Enable interrupts
```

```
; *****
;   P R O G R A M   L O O P
; *****
```

Loop:

```
    nop                   ; no operation = pause
    cbi PORTB, 0          ; turn off PB0
    nop                   ; no operation = pause
    cbi PORTB, 1          ; turn off PB1
    nop                   ; no operation = pause
    cbi PORTB, 2          ; turn off PB2
    nop                   ; no operation = pause
    cbi PORTB, 3          ; turn off PB3
    nop                   ; no operation = pause
    sbi PORTB, 0          ; turn on PB0
    nop                   ; no operation = pause
    sbi PORTB, 1          ; turn on PB1
    nop                   ; no operation = pause
    sbi PORTB, 2          ; turn on PB2
    nop                   ; no operation = pause
    sbi PORTB, 3          ; turn on PB3
    nop                   ; no operation = pause
```

```
    rjmp Loop
```

```
; End of source code
```

| CBR - Clear Bits in Register ( <a href="#">contents</a> )                                                                                                                                                              | SBR- Set Bits in Register ( <a href="#">contents</a> )                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Description:</p> <p>Clears the specified bits in register Rd. Performs the logical AND between the contents of register Rd and the complement of the constant mask K. The result will be placed in register Rd.</p> | <p>Description:</p> <p>Sets specified bits in register Rd. Performs the logical ORI between the contents of register Rd and constant mask K and places the result in the destination register Rd.</p> |

```
; *****
;   M A I N   P R O G R A M   I N I T
; *****
```

Main:

```
    ldi rmp, Low(RAMEND)
    out SPL, rmp           ; Init LSB stack pointer
    sei                   ; Enable interrupts
```

```
; *****
;   P R O G R A M   L O O P
; *****
```

Loop:

```
    ldi r16, 0b00101111    ; load 47 into r16 (0x2F)
    nop                   ; no operation = pause
    cbr r16, 0b00001000    ; (r16 = 0b0010111 = 0x27)
    nop                   ; no operation = pause
    cbr r16, 0b00000100    ; (r16 = 0b0010011 = 0x23)
    nop                   ; no operation = pause
    sbr r16, 0b00001000    ; (r16 = 0b0011001 = 0x2B)
    nop                   ; no operation = pause
    sbr r16, 0b01000000    ; (r16 = 0b0111001 = 0x6B)
    nop                   ; no operation = pause
    ldi r16, 0b00000000    ; clear r16
    nop                   ; no operation = pause
```

```
    rjmp Loop
```

```
; End of source code
```

**BCLR/BSET** works for all SREG, so it is not necessary to learn or use all of these alias instructions:

### SREG – Status Register

|               |     |     |     |     |     |     |     |     |
|---------------|-----|-----|-----|-----|-----|-----|-----|-----|
| Bit           | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| 0x3F          | I   | T   | H   | S   | V   | N   | Z   | C   |
| Read/Write    | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Initial Value | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |

BSET 0 SEC Set Carry  $C \leftarrow 1$   
BCLR 0 CLC Clear Carry  $C \leftarrow 0$   
BSET 1 SEZ Set Zero Flag  $Z \leftarrow 1$   
BCLR 1 CLZ Clear Zero Flag  $Z \leftarrow 0$   
BSET 2 SEN Set Negative Flag  $N \leftarrow 1$   
BCLR 2 CLN Clear Negative Flag  $N \leftarrow 0$   
BSET 3 SEV Set Twos Complement Overflow.  $V \leftarrow 1$   
BCLR 3 CLV Clear Twos Complement Overflow  $V \leftarrow 0$   
BSET 4 SES Set Signed Test Flag  $S \leftarrow 1$   
BCLR 4 CLS Clear Signed Test Flag  $S \leftarrow 0$   
BSET 5 SEH Set Half Carry Flag in SREG  $H \leftarrow 1$   
BCLR 5 CLH Clear Half Carry Flag in SREG  $H \leftarrow 0$   
BSET 6 SET Set T in SREG  $T \leftarrow 1$   
BCLR 6 CLT Clear T in SREG  $T \leftarrow 0$   
BSET 7 SEI Global Interrupt Enable  $I \leftarrow 1$   
BCLR 7 CLI Global Interrupt Disable  $I \leftarrow 0$

## COM - One's Complement ([contents](#))

### Description:

This instruction performs a one's complement of register Rd.

One's complement inverts all bits in a binary structure. 101 becomes 010.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    ldi r16, 0b00011111        ; pins to output  
    out DDRB, r16              ; and make it so  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    ldi r16, 0b00001101        ; turn on PB0, PB2, PB3  
    out PORTB, r16             ; and make it so  
    nop                        ; no operation = pause  
  
    com r16                    ; reverse r16 (0b11110010)  
    cbr r16, 0b11100000        ; upper bits cleared  
    out PORTB, r16             ; turn on PB1, PB4  
    nop                        ; no operation = pause  
  
    rjmp Loop
```

```
; End of source code
```



## CP - Compare ([contents](#))

### Description:

This instruction performs a compare between two registers Rd and Rr. None of the registers are changed. All conditional branches can be used after this instruction.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp           ; Init LSB stack pointer  
    ldi r17, 22            ; load 22 = 0x16 = 010110  
    ldi r18, 36            ; load 36 = 0x24 = 100100  
    sei                   ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    dec r18                ; decrease r18 by 1  
    cp r18, r17            ; compare the registers  
    brbc 1, Loop           ; branch if not equal  
  
    ldi r17, 22            ; load 22 = 0x16 = 010110  
    ldi r18, 36            ; load 36 = 0x24 = 100100  
  
    rjmp Loop
```

```
; End of source code
```

## CPC - Compare with Carry ([contents](#))

### Description:

This instruction performs a compare between two registers Rd and Rr and also takes into account the previous carry. None of the registers are changed. All conditional branches can be used after this instruction.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    sei                        ; Enable interrupts  
    .equ fred = 5573           ; define fred 0b0001010111000101 = 0x15C5  
    .equ barney = 4210         ; define barney 0b0001000001110010 = 0x1072  
    ldi r20, high(fred)        ; 0x15  
    ldi r19, low(fred)         ; 0xC5  
    ldi r18, high(barney)      ; 0x10  
    ldi r17, low(barney)       ; 0x72  
    ldi r16, 0b00000011       ; output for PB0, PB1  
    out DDRB, r16              ; make it so
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    cp r20, r18                ; compare upper bits  
    cpc r19, r17                ; compare lower bits  
    brbs 2, Fliptwo             ; branch if less than
```

Flipone: ; swap the numbers

```
    ldi r20, high(fred)  
    ldi r19, low(fred)  
    ldi r18, high(barney)  
    ldi r17, low(barney)  
    ldi r16, 0b00000001        ; turn on PB0  
    out PORTB, r16             ; make it so  
    rjmp Loop
```

Fliptwo: ; swap the numbers

```
    ldi r20, high(barney)  
    ldi r19, low(barney)  
    ldi r18, high(fred)  
    ldi r17, low(fred)  
    ldi r16, 0b00000010        ; turn on PB1  
    out PORTB, r16             ; make it so  
    rjmp Loop
```

; End of source code

## CPI - Compare with Immediate ([contents](#))

### Description:

This instruction performs a compare between register Rd and a constant. The register is not changed. All conditional branches can be used after this instruction.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 39              ; 0b00100111 = 0x27  
    .equ barney = 30            ; 0b00011110 = 0x1E  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    ldi r17, fred              ; the higher number here
```

Countdown:

```
    dec r17                    ; decrement by 1  
    cpi r17, barney            ; compare with 30  
    brbc 1, Countdown          ; if not equal keep counting down  
  
    rjmp Loop
```

```
; End of source code
```

## CPSE - Compare Skip if Equal ([contents](#))

### Description:

This instruction performs a compare between two registers Rd and Rr, and skips the next instruction if Rd = Rr.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred    = 87          ; (dec 87 = 0x57)  
    .equ barney  = 71          ; (dec 71 = 0x47)  
    ldi r17, fred              ; load 87 into r17  
    sei                          ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    ldi r16, barney            ; load 71 into r16
```

Countdown:

```
    inc r16                    ; increase r16 by 1  
    cpse r17, r16              ; compare and skip next if equal  
    rjmp Countdown             ; not equal so loop to decrement  
    rjmp Loop                  ; equal so start again
```

```
; End of source code
```

## DEC – Decrement ([contents](#))

### Description

Subtracts one -1- from the contents of register Rd and places the result in the destination register Rd.

The C Flag in SREG is not affected by the operation, thus allowing the DEC instruction to be used on a loop counter in multiple-precision computations.

When operating on unsigned values, only BREQ and BRNE branches can be expected to perform consistently. When operating on two's complement values, all signed branches are available.

```
; *****  
; M A I N P R O G R A M I N I T  
; *****
```

Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp                ; Init LSB stack pointer  
    .equ fred = 87              ; (dec 87 = 0x57)  
    .equ barney = 71            ; (dec 71 = 0x47)  
    ldi r17, barney             ; load 71 into r17  
    sei                        ; Enable interrupts
```

```
; *****  
; P R O G R A M L O O P  
; *****
```

Loop:

```
    ldi r16, fred               ; load 87 into r16
```

Countdown:

```
    dec r16                    ; decrease r16 by 1  
    cp r17, r16                ; compare numbers  
    brbc 1, Countdown          ; not equal so keep counting  
    rjmp Loop                  ; if equal loop and reset
```

```
; End of source code
```

## EOR – Exclusive OR ([contents](#))

### Description

Performs the logical EOR between the contents of register Rd and register Rr and places the result in the destination register Rd.

**XOR Truth Table**

| Inputs |   | Output           |
|--------|---|------------------|
| A      | B | $Y = A \oplus B$ |
| 0      | 0 | 0                |
| 0      | 1 | 1                |
| 1      | 0 | 1                |
| 1      | 1 | 0                |

```
; *****
; M A I N P R O G R A M I N I T
; *****

Main:
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer
    .equ fred = 0b00101101    ; 45 = 0x2D
    .equ barney = 0b01100100  ; 100 = 0x64
    ldi r18, barney            ; load 100 into r18
    sei                        ; Enable interrupts

; *****
; P R O G R A M L O O P
; *****

Loop:
    clr r19                    ; clear r20
    ldi r17, fred              ; load 45 into r17
    eor r17, r18               ; exclusive or r17 ^ r18

; next we do XOR again via an expression, with the same
; result () put into r19
; 00101101 XOR
; 01100100 =
; -----
; 01001001 (73 = 0x49)

    ldi r19, (barney ^ fred)   ; immediate XOR expression
    nop                        ; no operation = pause
    rjmp Loop

; End of source code
```

## ICALL – Indirect Call to Subroutine ([contents](#))

### Description

Calls to a subroutine within the entire 4M (words) Program memory. The return address (to the instruction after the CALL) will be stored onto the Stack. See also RCALL. The Stack Pointer uses a post-decrement scheme during CALL. Probably not that useful for the attiny13A (1k memory).

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 0x3e           ; fred = 62  
    .equ barney = 0x11         ; barney = 17  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    ldi r16, fred              ; load 45 (0x2D) into r16
```

MaketheCall:

```
; these next two instructions also work in this context if put in Main  
; and executed only once but they are repeated here each time for clarity
```

```
    ldi z1, low(Justdecrement) ; load low part of subroutine address  
                                ; into r30 (Z low byte)  
    ldi zh, high(Justdecrement) ; load high part of subroutine address  
                                ; into r31 (Z high byte)  
    icall                       ; indirect address the subroutine  
    cpi r16, barney             ; compare with 17  
    brbc 1, MaketheCall         ; if not equal keep counting down  
  
    rjmp Loop                   ; equal so start again
```

Justdecrement:

```
    dec r16                    ; decrease by one  
    ret
```

```
; End of source code
```

## IJMP - Indirect Jump ([contents](#))

### Description

Indirect jump to the address pointed to by the Z (16 bits) pointer register in the register file. The Z pointer register is 16 bits wide and allows jump within the lowest 64K words (128K bytes) section of program memory. Probably not that useful for the attiny13A (1k memory).

```
; *****
;   M A I N   P R O G R A M   I N I T
; *****

Main:
    ldi rmp, Low(RAMEND)
    out SPL, rmp                ; Init LSB stack pointer
    .equ fred = 0x2E            ; fred = 46
    .equ barney = 0x11          ; barney = 17
    sei                        ; Enable interrupts

; *****
;   P R O G R A M   L O O P
; *****

Loop:
    ldi r16, fred                ; load 46 (0x2E) into r16

Compare:
    cpi r16, barney              ; compare r16 to 0x11
    brbs 1, Loop                ; if equal then Loop

; else we decrement using ijmp

    ldi z1, low(Justdecrement)   ; load low part of jump address
                                ; into r30 (Z low byte)
    ldi zh, high(Justdecrement)  ; load high part of jump address
                                ; into r31 (Z high byte)
    ijmp                        ; indirect jump

Justdecrement:
    dec r16                      ; decrease by one
    ldi z1, low(Compare)         ; load low part of jump address
    ldi zh, high(Compare)        ; load high part of jump address
    ijmp                        ; indirect jump

; End of source code
```



|                                                                                                                                                                                                                             |                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IN - Load an I/O Location to Register</b><br><b>(<a href="#">contents</a>)</b><br><br>Description:<br>Loads data from the I/O Space (Ports, Timers, Configuration registers etc.) into register Rd in the register file. | <b>OUT - Store Register to I/O Location</b><br><b>(<a href="#">contents</a>)</b><br><br>Description:<br>Stores data from register Rr in the register file to I/O Space (Ports, Timers, Configuration registers etc.). |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
; *****
;  M A I N    P R O G R A M    I N I T
;  *****
```

Main:

```
    ldi rmp, Low(RAMEND)
    out SPL, rmp                ; Init LSB stack pointer
    .equ lights = 0b00001111   ; output lights
    ldi r16, lights             ; PB0-PB3 set to output
    ldi r17, lights             ; for eor bit manipulation later
    out DDRB, r16               ; make it so in DDRB
    out PORTB, r16              ; and turn them on to start with
    sei                         ; Enable interrupts
```

```
; *****
;    P R O G R A M    L O O P
;  *****
```

Loop:

```
    in r16, PINB                ; status of the pins to r16
    eor r16, r17                ; flip the lights
    out PORTB, r16              ; and write them
    nop                         ; no operation = pause

    rjmp Loop                   ; rinse and repeat
```

```
; End of source code
```

## INC - Increment ([contents](#))

### Description:

Adds one to the contents of register Rd and places the result in the destination register Rd. The C flag in SREG is not affected by the operation, thus allowing the INC instruction to be used on a loop counter in multiple-precision computations. When operating on unsigned numbers, only BREQ and BRNE branches can be expected to perform consistently. When operating on two's complement values, all signed branches are available.

```
; *****  
; M A I N P R O G R A M I N I T  
; *****
```

### Main:

```
ldi rmp,Low(RAMEND)  
out SPL,rmp          ; Init LSB stack pointer  
.equ fred = 87       ; (dec 87 = 0x57)  
.equ barney = 71      ; (dec 71 = 0x47)  
ldi r17, fred        ; load 87 into r17  
sei                  ; Enable interrupts
```

```
; *****  
; P R O G R A M L O O P  
; *****
```

### Loop:

```
ldi r16, barney      ; load 71 into r16
```

### Countup:

```
inc r16              ; increase r16 by 1  
cp r17, r16          ; compare numbers  
brbc 1, Countup      ; not equal so keep counting  
nop                  ; no operation = pause  
  
rjmp Loop            ; if equal loop and reset
```

```
; End of source code
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LD - Load Indirect from data space to Register using Index X (<a href="#">contents</a>)</b></p> <p>Description:<br/>Loads one byte indirect from the data space to a register. For parts with SRAM, the data space consists of the register file, I/O memory and internal SRAM (and external SRAM if applicable). For parts without SRAM, the data space consists of the register file only. The EEPROM has a separate address space.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <p><b>ST - Store Indirect From Register to data space using Index X (<a href="#">contents</a>)</b></p> <p>Description:<br/>Stores one byte indirect from a register to data space. For parts with SRAM, the data space consists of the register file, I/O memory and internal SRAM (and external SRAM if applicable). For parts without SRAM, the data space consists of the register file only. The EEPROM has a separate address space.</p> |
| <p>The data location is pointed to by the X (16 bits) pointer register in the register file. Memory access is limited to the current data segment of 64K bytes. To access another data segment in devices with more than 64K bytes data space, the RAMPX, RAMPY, RAMPZ in register in the I/O area has to be changed.</p> <p>The X pointer register can either be left unchanged by the operation, or it can be post-incremented or pre-decremented. These features are especially suited for accessing arrays, tables, and stack pointer usage of the X pointer register. Note that only the low byte of the X pointer is updated in devices with no more than 256 bytes data space. For such devices, the high byte of the pointer is not used by this instruction and can be used for other purposes. The RAMPX register in the I/O area is updated in parts with more than 64K bytes data space or more than 64K bytes program memory, and the increment/decrement is added to the entire 24-bit address on such devices.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                               |

|                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>.dseg    ; data section MyArray: .BYTE 4 ; 4 bytes allocated  ; ***** ;   M A I N   P R O G R A M   I N I T ; ***** ;  Main:     ldi rmp,Low(RAMEND)     out SPL,rmp ; Init stack pointer     sei ; Enable interrupts ; ; ***** ;   P R O G R A M   L O O P ; *****  Loop:     clr r16      ; clear registers     clr r17     clr r18     clr r19     clr r20</pre> | <pre>    ; load address of array     ldi Xl, low(MyArray)     ldi Xh, high(MyArray)      ldi r16, 155    ; load 0x9B     st X+, r16      ; and store it     ldi r16, 32     ; load 0x20     st X+, r16      ; and store it     ldi r16, 105    ; load 0x69     st X+, r16      ; and store it     ldi r16, 67     ; load 0x43     st X, r16       ; and store it      ; load address of array     ldi Xl, low(MyArray)     ldi Xh, high(MyArray)      ld r17, X+       ; retrieve 0x9B     ld r18, X+       ; retrieve 0x20     ld r19, X+       ; retrieve 0x69     ld r20, X        ; retrieve 0x43     rjmp Loop  ; End of source code</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| LDS - Load Direct from data space ( <a href="#">contents</a> )                                                                                                                                                                                                                                                                      | STS - Store Direct to data space ( <a href="#">contents</a> )                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Description:</p> <p>Loads one byte from the data space to a register. For parts with SRAM, the data space consists of the register file, I/O memory and internal SRAM (and external SRAM if applicable). For parts without SRAM, the data space consists of the register file only. The EEPROM has a separate address space.</p> | <p>Description:</p> <p>Stores one byte from a Register to the data space. For parts with SRAM, the data space consists of the register file, I/O memory and internal SRAM (and external SRAM if applicable). For parts without SRAM, the data space consists of the register file only. The EEPROM has a separate address space.</p> |
| <p>A 16-bit address must be supplied. Memory access is limited to the current data segment of 64K bytes. The LDS instruction uses the RAMPD register to access memory above 64K bytes. To access another data segment in devices with more than 64K bytes data space, the RAMPD in register in the I/O area has to be changed.</p>  |                                                                                                                                                                                                                                                                                                                                      |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>.dseg MyArray: .BYTE 4 ; 4 bytes allocated  Main:     ldi rmp,Low(RAMEND)     out SPL,rmp ; Init stack pointer     sei ; Enable interrupts  ; ***** ;   P R O G R A M   L O O P ; *****  Loop:     clr r16 ; clear registers     clr r17     clr r18     clr r19      ; load then store 0x9B     ldi r16, 155     sts (MyArray), r16      ; load then store 0x20 (note     ; address increment +1)     ldi r16, 32     sts (MyArray)+1, r16      ; load then store 0x69 (note     ; address increment +2)      ldi r16, 105     sts (MyArray)+2, r16</pre> | <pre>; load then store 0x43 (note ; address increment +3)  ldi r16, 67 sts (MyArray)+3, r16  clr r16 ; clear r16  ; load back and check all ; the values to verify ; the store/load cycle  lds r16, (MyArray)  lds r17, (MyArray)+1  lds r18, (MyArray)+2  lds r19, (MyArray)+3  rjmp Loop  ; End of source code</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## LPM - Load Program Memory ([contents](#))

### Description:

Loads one byte pointed to by the Z register into the destination register Rd. This instruction features a 100% space effective constant initialization or constant data fetch. The program memory is organized in 16 bit words while the Z pointer is a byte address. Thus, the least significant bit of the Z pointer selects either low byte (ZLSB = 0) or high byte (ZLSB = 1). This instruction can address the first 64K bytes (32K words) of program memory. The Z pointer register can either be left unchanged by the operation, or it can be incremented. The increment does not apply to the RAMPX, RAMPY, RAMPZ register.

Devices with Self-Programming capability can use the LPM instruction to read the fuse and lock bit values. Refer to the device documentation for a detailed description.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

### Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp                ; Init LSB stack pointer  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

### Loop:

```
    ldi Zl, low(MyArray*2)      ; load low part of subroutine address  
                                ; into r30 (Z low byte)  
    ldi Zh, high(MyArray*2)     ; load high part of subroutine address  
                                ; into r31 (Z high byte)  
    lpm r17, Z+                 ; load first byte and increment  
    lpm r18, Z+                 ; load second byte and increment  
    lpm r19, Z+                 ; load third byte and increment  
    lpm r20, Z                  ; load last byte  
    nop                        ; no operation = pause  
    clr r17                     ; clear all registers  
    clr r18  
    clr r19  
    clr r20  
    rjmp Loop
```

### MyArray:

```
    .DB 5, 11, 204, 6           ; four 1-byte values
```

```
; End of source code
```

|                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LSL - Logical Shift Left (<a href="#">contents</a>)</b><br><br>Description:<br>Shifts all bits in Rd one place to the left. Bit 0 is cleared. Bit 7 is loaded into the C flag of the SREG. This operation effectively multiplies signed and unsigned values by two. | <b>LSR - Logical Shift Right (<a href="#">contents</a>)</b><br><br>Description:<br>Shifts all bits in Rd one place to the right. Bit 7 is cleared. Bit 0 is loaded into the C flag of the SREG. This operation effectively divides an unsigned value by two. The C flag can be used to round the result. |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
; *****
;  M A I N    P R O G R A M    I N I T
;  *****
```

Main:

```
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer
    .equ fred = 0b00000011     ; define 3 = 0x03
    sei                        ; Enable interrupts
```

```
; *****
;   P R O G R A M    L O O P
;  *****
```

Loop:

```
    ldi r16, fred              ; load 0x03 into r16
    lsl r16                    ; 2 x 3 = 6 = 0x06
    lsl r16                    ; 2 x 6 = 12 = 0x0C
    lsl r16                    ; 2 x 12 = 24 = 0x18

    nop                        ; no operation = pause

    lsr r16                    ; 24 ÷ 2 = 12 = 0x0C
    lsr r16                    ; 12 ÷ 2 = 6 = 0x06
    lsr r16                    ; 6 ÷ 2 = 3 = 0x03
    clr r16
```

```
    rjmp Loop
```

```
; End of source code
```

## MOV - Copy Register ([contents](#))

### Description:

This instruction makes a copy of one register into another. The source register Rr is left unchanged, while the destination register Rd is loaded with a copy of Rr.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 0b00111010    ; define 58 = 0x3A  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    clr r20                    ; clear destination register  
    ldi r16, fred              ; load 0x3A to r16  
    mov r17, r16               ; move 0x3A to r17  
    clr r16                     ; clear r16  
    mov r18, r17               ; move 0x3A to r18  
    clr r17                     ; clear r17  
    mov r19, r18               ; move 0x3A to r19  
    clr r18                     ; clear r18  
    mov r20, r19               ; move 0x3A to r20  
    clr r19                     ; clear r19  
    nop                        ; no operation = pause
```

```
    rjmp Loop
```

```
; End of source code
```

## MOVW - Copy Register Word ([contents](#))

### Description:

This instruction makes a copy of one register pair into another register pair. The source register pair Rr+1:Rr is left unchanged, while the destination register pair Rd+1:Rd is loaded with a copy of Rr + 1:Rr.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 5124           ; 5124 = 0x1404 = 0b0001 0100 0000 0100  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    clr r22                    ; clear destination registers  
    clr r23                    ; clear destination registers  
    ldi r16, low(fred)         ; load 0x04 to r16  
    ldi r17, high(fred)        ; load 0x14 to r17  
    movw r18, r16              ; move 5124 up two registers  
    clr r16                    ; clear r16  
    clr r17                    ; clear r17  
    movw r20, r18              ; move 5124 up two registers  
    clr r18                    ; clear r18  
    clr r19                    ; clear r19  
    movw r22, r20              ; move 5124 up two registers  
    clr r20                    ; clear r20  
    clr r21                    ; clear r21  
    nop                        ; no operation = pause
```

```
    rjmp Loop
```

```
; End of source code
```



## NEG - Two's Complement ([contents](#))

### Description:

Replaces the contents of register Rd with its two's complement; the value \$80 is left unchanged.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 22             ; set fred 0x16  
    .equ barney = 42           ; set barney 0x2A  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

Loop:

```
    ldi r16, fred              ; load 22 to r16  
    ldi r17, barney            ; load 42 to r17  
    sub r16, r17               ; subtract to get 0xEC (236)  
    neg r16                    ; NEG to find absolute value (20 = 0x14)  
    clr r16                    ; clear registers  
    clr r17  
    nop                        ; no operation = pause  
    rjmp Loop
```

; End of source code

### How does it work?

$22 - 42 = -20 = 0xEC = 0b11101100 = -128 + 64 + 32 + 8 + 4$  (first bit is negative sign)

| sign | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|------|----|----|----|---|---|---|---|
| 1    | 1  | 1  | 0  | 1 | 1 | 0 | 0 |

### Two's complement algorithm

Flip all bits =  $0b00010011$

Add one =  $0b00010100 = 16 + 4 = 20 = 0x14$

| sign | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|------|----|----|----|---|---|---|---|
| 0    | 0  | 0  | 1  | 0 | 1 | 0 | 0 |

## NOP - No Operation ([contents](#))

Description: This instruction performs a single cycle No Operation.

See previous pages.

## OR - Logical OR ([contents](#))

Description: Performs the logical OR between the contents of register Rd and register Rr and places the result in the destination register Rd.

OR Truth Table

| Inputs |   | Output  |
|--------|---|---------|
| A      | B | Y = A+B |
| 0      | 0 | 0       |
| 0      | 1 | 1       |
| 1      | 0 | 1       |
| 1      | 1 | 1       |

|            |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|
| r16        | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 |
| r17        | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| r16 OR r17 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 |

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 0b01011100    ; set fred (0x5D)  
    .equ barney = 0b00000001   ; set barney  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

Loop:

```
    ldi r16, fred              ; load 0x5C to r16  
    ldi r17, barney           ; load 0x01 to r17  
    or r16, r17               ; perform logical OR (0x5D)  
    clr r16                   ; clear registers  
    clr r17  
    rjmp Loop
```

```
; End of source code
```

## ORI - Logical OR with Immediate ([contents](#))

### Description:

Performs the logical OR between the contents of register Rd and a constant and places the result in the destination register Rd.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 0b01011100    ; set fred (0x5D)  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

Loop:

```
    ldi r16, fred              ; load 0x5C to r16  
    ori r16, 0x03             ; perform logical OR (0x5F)  
    clr r16                   ; clear registers  
    rjmp Loop
```

```
; End of source code
```

## OUT - Store Register to I/O Location ([contents](#))

### Description:

Stores data from register Rr in the register file to I/O Space (Ports, Timers, Configuration registers etc.).

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp                ; Init LSB stack pointer  
    ldi r16, 0b00001111        ; Four ports to output  
    out DDRB, r16              ; and make it so  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    ldi r16, 0b00001010        ; turn on PB1 and PB3  
    out PORTB, r16             ; and make it so  
    nop                        ; no operation = pause  
    ldi r16, 0b00000101        ; turn on PB0 and PB2  
    out PORTB, r16             ; and make it so  
    nop                        ; no operation = pause
```

```
    rjmp Loop
```

```
; End of source code
```

**PUSH - Push Register on Stack ([contents](#))****Description:**

This instruction stores the contents of register Rr on the STACK. The stack pointer is post-decremented by 1 after the PUSH.

**POP - Pop Register from Stack ([contents](#))****Description:**

This instruction loads register Rd with a byte from the Stack. The stack pointer is pre-incremented by 1 before the POP.

```
; *****
;   M A I N   P R O G R A M   I N I T
;   *****
```

Main:

```
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer
    .equ fred = 13             ; define fred 0x0D
    .equ barney = 27            ; define barney 0x1B
    .equ wilma = 78             ; define wilma 0x4E
    .equ betty = 213            ; define betty 0xD5
    ldi r16, fred               ; load into register
    ldi r17, barney             ; load into register
    ldi r18, wilma              ; load into register
    ldi r19, betty              ; load into register
    sei                         ; Enable interrupts
```

```
; *****
;   P R O G R A M   L O O P
;   *****
```

Loop:

```
    push r16                   ; push fred to the stack
    push r17                   ; push barney to the stack
    push r18                   ; push wilma to the stack
    push r19                   ; push betty to the stack
    clr r16                    ; clear registers
    clr r17
    clr r18
    clr r19
    nop                        ; no operation = pause
    pop r19                    ; back off the stack in reverse
    pop r18
    pop r17
    pop r16

    rjmp Loop
```

```
; End of source code
```

## RCALL - Relative Call to Subroutine ([contents](#))

### Description:

Relative call to an address within  $PC - 2K + 1$  and  $PC + 2K$  (words). The return address (the instruction after the RCALL) is stored onto the stack. (See also CALL). In the assembler, labels are used instead of relative operands. For AVR microcontrollers with program memory not exceeding 4K words (8K bytes) this instruction can address the entire memory from every address location. The stack pointer uses a post-decrement scheme during RCALL.

RCALL uses less flash (1 word) than CALL (2 words). These two instructions are the same with one difference: RCALL uses 1 word ROM (and because of this, it has a limited address-range), while CALL uses 2 words ROM. Usually, you use RCALL for loops, as you can save some program-memory this way.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 10             ; initialise 10 = 0x0A  
    ldi r16, fred              ; load fred into r16  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

Loop:

```
    mov r17, r16                ; move counter to r17  
    rcall Countdown             ; call the subroutine  
    rjmp Loop
```

Countdown:

```
    dec r17                     ; decrease by 1  
    brbc 1, Countdown           ; if not zero then continue countdown  
    ret                         ; else return to main loop
```

```
; End of source code
```

## RET - Return from Subroutine ([contents](#))

### Description:

Returns from subroutine. The return address is loaded from the Stack. The stack pointer uses a pre-increment scheme during RET.

See previous pages.

## RETI - Return from Interrupt ([contents](#))

### Description:

Returns from interrupt. The return address is loaded from the Stack and the global interrupt flag is set.

Note that the status register is not automatically stored when entering an interrupt routine, and it is not restored when returning from an interrupt routine. This must be handled by the application program. The stack pointer uses a pre-increment scheme during RETI. Here is a program that uses RETI:

## Interval – making a real program ([contents](#))

Note that this will bring in some other instructions not covered yet, but surely we are ready for a change up?

Aim: to make an Attiny13a assembler program that sleeps most of the time, but wakes up every 2 seconds and flips an LED on or off.

- define some constants such as pins and ports
- set up the watchdog interrupt jump
- code the interrupt routine (in this case a watchdog timer)
- setup up ports and the watchdog timer register
- code the loop which sleeps most of the time, reducing power needs of the microcontroller

```
;
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 / _____ | 8
; RESET o--|         |--o VCC
;      2 |         | 7
; PB3  o--|         |--o PB2
;      3 |         | 6
; PB4  o--|         |--o PB1
;      4 |         | 5
; GND  o--|         |--o PB0
;      | _____ |
;
; continued over the page
```

```

.nolist
.include "tn13adef.inc"          ; Define device ATtiny13A
.list

; *****
;       H A R D W A R E
; *****

; *****
;   P O R T S   A N D   P I N S
; *****

.equ pOut = PORTB                ; Define a output port
.equ pDir = DDRB                 ; Define a direction port
.equ pIn = PINB                  ; Define an input port
.equ bMyPinO = PORTB0            ; Define an output pin
.equ bDir0 = DDB0                ; Define a direction pin
.equ bMyIn = PINB0               ; Define an input pin

; *****
;   A D J U S T A B L E   C O N S T
; *****

.equ clock=1200000               ; Define clock frequency

; *****
;   F I X   &   D E R I V .   C O N S T
; *****
;
.equ cTc0Clk = clock / 256       ; Define from clock

; *****
;       R E G I S T E R S
; *****

; free: R0 to R14
.def rSreg = R15                 ; Save/Restore status port
.def rmp = R16                   ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...

; *****
;       S R A M
; *****

.dseg
.org SRAM_START
sLabel1:
.byte 16 ; Reserve 16 bytes

; continued over the page

```



```

; *****
;           C O D E
; *****

.cseg
.org 000000

; *****
; R E S E T   &   I N T - V E C T O R S
; *****
    rjmp Main                ; Reset vector
    reti ; INT0
    reti ; PCIO
    reti ; OVFO
    reti ; ERDY
    reti ; ACI
    reti ; OC0A
    reti ; OC0B
    rjmp mytimer_ISR        ; watchdog timer interrupt
    reti ; ADCC

; *****
;   I N T - S E R V I C E   R O U T .
; *****

; Add all interrupt service routines

mytimer_ISR:

    cli                    ; clear interrupt flag
    push r16               ; save global registers on stack
    push r17
    push r18
    in r16, SREG            ; save SREG
    push r16               ; do this if you want to use
                           ; r16 in your interrupt routine

    ; here is the interrupt code - turn on or off a LED

    ldi r17, 0b00000001    ; load r17 with the first bit on
    in r16, PINB            ; load the current state of the pins
    eor r16, r17            ; flip the first bit (if on, off etc)
    out PORTB, r16         ; alter the pins

    pop r16                 ; do this if you pushed SREG above
    out SREG, r16           ; restore SREG
    pop r18                 ; restore global registers
    pop r17
    pop r16
    sei                    ; set interrupt pin
    reti                  ; return from interrupt

; continued over the page

```

```
; *****
;   M A I N   P R O G R A M   I N I T
; *****
```

Main:

```
ldi rmp, Low(RAMEND)
out SPL, rmp                ; Init LSB stack pointer

ldi r16, 0b00000001        ; output mode for PB0
out DDRB, r16              ; and make it so
```

```
; 2 second timer
```

| Bit           | 7     | 6     | 5    | 4    | 3   | 2    | 1    | 0    |       |
|---------------|-------|-------|------|------|-----|------|------|------|-------|
| 0x21          | WDTIF | WDTIE | WDP3 | WDCE | WDE | WDP2 | WDP1 | WDP0 | WDTCR |
| Read/Write    | R/W   | R/W   | R/W  | R/W  | R/W | R/W  | R/W  | R/W  |       |
| Initial Value | 0     | 0     | 0    | 0    | X   | 0    | 0    | 0    |       |

**Table 8-2.** Watchdog Timer Prescale Select

| WDP3 | WDP2 | WDP1 | WDP0 | Number of WDT Oscillator Cycles | Typical Time-out at $V_{CC} = 5.0V$ |
|------|------|------|------|---------------------------------|-------------------------------------|
| 0    | 0    | 0    | 0    | 2K (2048) cycles                | 16 ms                               |
| 0    | 0    | 0    | 1    | 4K (4096) cycles                | 32 ms                               |
| 0    | 0    | 1    | 0    | 8K (8192) cycles                | 64 ms                               |
| 0    | 0    | 1    | 1    | 16K (16384) cycles              | 0.125 s                             |
| 0    | 1    | 0    | 0    | 32K (32768) cycles              | 0.25 s                              |
| 0    | 1    | 0    | 1    | 64K (65536) cycles              | 0.5 s                               |
| 0    | 1    | 1    | 0    | 128K (131072) cycles            | 1.0 s                               |
| 0    | 1    | 1    | 1    | 256K (262144) cycles            | 2.0 s                               |
| 1    | 0    | 0    | 0    | 512K (524288) cycles            | 4.0 s                               |
| 1    | 0    | 0    | 1    | 1024K (1048576) cycles          | 8.0 s                               |

```
ldi r16, 0b01000111        ; set WDTIE, WDP2, WDP1 and WDP0
out WDTCR, r16              ; and out to WDTCR
```

```
sei                          ; Enable interrupts
```

; continued over the page

```
; *****
;   P R O G R A M   L O O P
; *****
```

Loop:

```
; sleep enable, power down mode
```

| Bit           | 7 | 6   | 5   | 4   | 3   | 2 | 1     | 0     |       |
|---------------|---|-----|-----|-----|-----|---|-------|-------|-------|
| 0x35          | - | PUD | SE  | SM1 | SM0 | - | ISC01 | ISC00 | MCUCR |
| Read/Write    | R | R/W | R/W | R/W | R/W | R | R/W   | R/W   |       |
| Initial Value | 0 | 0   | 0   | 0   | 0   | 0 | 0     | 0     |       |

**Table 7-2.** Sleep Mode Select

| SM1 | SM0 | Sleep Mode          |
|-----|-----|---------------------|
| 0   | 0   | Idle                |
| 0   | 1   | ADC Noise Reduction |
| 1   | 0   | Power-down          |
| 1   | 1   | Reserved            |

```
ldi r16, 0b00110000      ; set SE and SM1
out MCUCR, r16           ; and out to MCUCR
sleep                    ; and sleep until interrupted
rjmp Loop
```

```
; End of source code
```

Some comments about the previous program:

The use of the interrupt vectors is important, you need to know which is which - for example the timer overflow is not the same as the watchdog timer. Here they are again, from the datasheet:

| Vector No. | Program Address | Source     | Interrupt Definition                                          |
|------------|-----------------|------------|---------------------------------------------------------------|
| 1          | 0x0000          | RESET      | External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset |
| 2          | 0x0001          | INT0       | External Interrupt Request 0                                  |
| 3          | 0x0002          | PCINT0     | Pin Change Interrupt Request 0                                |
| 4          | 0x0003          | TIM0_OVF   | Timer/Counter Overflow                                        |
| 5          | 0x0004          | EE_RDY     | EEPROM Ready                                                  |
| 6          | 0x0005          | ANA_COMP   | Analog Comparator                                             |
| 7          | 0x0006          | TIM0_COMPA | Timer/Counter Compare Match A                                 |
| 8          | 0x0007          | TIM0_COMPB | Timer/Counter Compare Match B                                 |
| 9          | 0x0008          | WDT        | Watchdog Time-out                                             |
| 10         | 0x0009          | ADC        | ADC Conversion Complete                                       |

So in our code, when the watchdog timer provides an interrupt, the interrupt service routine must jump from 0x0008 as shown in “mytimer\_ISR”. As the main program is stopped, the ISR is best nested by commands that save registers and the SREG before the code is executed. If the code is quite long, it is recommended to maybe just switch a boolean flag and then continue with the main program. In our case it was just a flick of an LED, so only a few lines required.

The watchdog timer works by setting the conditions in the Watchdog Timer Control Register (WDTCR). Some code that I’ve seen in resources online uses notation such as:

```
ldi r16, (1<<WDTIE) | (1<<WDP2) | (1<<WDP1) | (1<<WDP0)
out WDTCR, r16
```

I have also see the binary converted to hexadecimal as follows...

```
ldi r16, 0x47      ;(or decimal 71)
out WDTCR, r16
```

These types of notation are clearly preferred by some, but I like to see the actual switches in binary so that the output aligns directly with the datasheet.

| WDTIF | WDTIE | WDP3 | WDCE | WDE | WDP2 | WDP1 | WDP0 |
|-------|-------|------|------|-----|------|------|------|
| 0     | 1     | 0    | 0    | 0   | 1    | 1    | 1    |

Which for me is easily represented as 0b01000111. Similarly, using the datasheet and reading about the settings for sleep in the MicroController Unit Control Register (MCUCR) the binary representation is as follows.

| – | PUD | SE | SM1 | SM0 | – | ISC01 | ISC00 |
|---|-----|----|-----|-----|---|-------|-------|
| 0 | 0   | 1  | 1   | 0   | 0 | 0     | 0     |

Which is loaded into r16 as 0b00110000 and then out to MCUCR.

## RJMP - Relative Jump ([contents](#))

Description:

Relative jump to an address within PC - 2K +1 and PC + 2K (words). In the assembler, labels are used instead of relative operands. For AVR microcontrollers with program memory not exceeding 4K words (8K bytes) this instruction can address the entire memory from every address location.

See previous pages.

## ROL - Rotate Left through Carry ([contents](#))

### Description:

Shifts all bits in Rd one place to the left. The C flag is shifted into bit 0 of Rd. Bit 7 is shifted into the C flag. This operation, combined with LSL, effectively multiplies multi-byte signed and unsigned values by two.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp           ; Init LSB stack pointer  
    .equ fred = 5         ; define fred = 0x101  
    .equ counter = 25     ; rol a few times (0x19)  
    sei                   ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

### Loop:

```
    ldi r17, counter      ; load counter  
    ldi r16, fred         ; load 0x05
```

### Rolalot:

```
    rol r16               ; rol r16  
    dec r17               ; counter decrement  
    brbc 1, Rolalot       ; if counter not zero, loop  
    rjmp Loop             ; start again
```

```
; End of source code
```

| Decimal | Binary   | Hexadecimal | SREG     | SREG                     |
|---------|----------|-------------|----------|--------------------------|
| 5       | 00000101 | 0x05        | 10000000 | interrupt                |
| 10      | 00001010 | 0x0A        | 10000000 |                          |
| 20      | 00010100 | 0x14        | 10100000 | half-carry               |
| 40      | 00101000 | 0x28        | 10000000 |                          |
| 80      | 01010000 | 0x50        | 10100000 | half-carry               |
| 160     | 10100000 | 0xA0        | 10001100 | $S = N \oplus V$         |
| 64      | 01000000 | 0x40        | 10001101 | $S = N \oplus V$ , carry |
| 128     | 10000001 | 0x81        | 10001100 | $S = N \oplus V$         |
| 2       | 00000010 | 0x02        | 10001101 | $S = N \oplus V$ , carry |
| 5       | 00000101 | 0x05        | 10000000 |                          |
| 10      | 00001010 | 0x0A        | 10000000 |                          |

## ROR - Rotate Right through Carry ([contents](#))

### Description:

Shifts all bits in Rd one place to the right. The C flag is shifted into bit 7 of Rd. Bit 0 is shifted into the C flag. This operation, combined with ASR, effectively divides multi-byte signed values by two. Combined with LSR it effectively divides multi-byte unsigned values by two. The carry flag can be used to round the result.

```
; *****
;   M A I N   P R O G R A M   I N I T
; *****

Main:
    ldi rmp,Low(RAMEND)
    out SPL,rmp           ; Init LSB stack pointer
    .equ fred = 5         ; define fred = 0x101
    .equ counter = 25     ; ror a few times (0x19)
    sei                   ; Enable interrupts

; *****
;   P R O G R A M   L O O P
; *****

Loop:
    ldi r17, counter      ; load counter
    ldi r16, fred         ; load 0x05

Rotalot:
    ror r16               ; ror r16
    dec r17               ; counter decrement
    brbc 1, Rotalot       ; if counter not zero, loop
    rjmp Loop             ; start again

; End of source code
```

| Decimal | Binary   | Hexadecimal | SREG     | SREG                     |
|---------|----------|-------------|----------|--------------------------|
| 5       | 00000101 | 0x05        | 10000000 | interrupt                |
| 2       | 00000010 | 0x02        | 10011001 | $S = N \oplus V$ , carry |
| 128     | 10000001 | 0x81        | 10001100 | N, V                     |
| 64      | 01000000 | 0x40        | 10011001 | $S = N \oplus V$ , carry |
| 160     | 10100000 | 0xA0        | 10001100 | N, V                     |
| 80      | 01010000 | 0x50        | 10000000 |                          |
| 40      | 00101000 | 0x28        | 10000000 |                          |
| 20      | 00010100 | 0x14        | 10000000 |                          |
| 10      | 00001010 | 0x0A        | 10000000 |                          |
| 5       | 00000101 | 0x05        | 10000000 |                          |
| 2       | 00000010 | 0x02        | 10011001 | $S = N \oplus V$ , carry |

## Multiplying binary Numbers ([contents](#))

The Attiny13a does not have the MUL, MULS or MULSU instructions, so here is a multiplying algorithm based on, among others, the following two sources:

1. [Textbook](#) “Some Assembly Required: Assembly Language Programming with the AVR Microcontroller” by Timothy S Margush (p438)
2. [Youtube](#) video entitled “Multiplication Using Add Shift Method” by Mr. Arnab Chakraborty

```
; *****
;   M A I N   P R O G R A M   I N I T
;   *****
```

Main:

```
    ldi rmp,Low(RAMEND)
    out SPL,rmp           ; Init LSB stack pointer
    .equ fred = 27        ; fred = 0x1B
    .equ barney = 58      ; barney = 0x3A
    sei                  ; Enable interrupts
```

```
; *****
;   P R O G R A M   L O O P
;   *****
```

Loop:

```
    clr r0                ; where most significant byte will be
    clr r1                ; where least significant byte will be
    ldi r16, fred         ; load multiplicand
    ldi r17, barney       ; load multiplier
    rcall multiply        ; recall multiplying routine
    rjmp Loop
```

multiply:

```
    ; result will be in r0:r1 (e.g. 0x061E or 1566 as 06:1E)
```

```
    ldi r18, 8            ;loop counter, 8 bits to check
```

next\_bit:

```
    lsr r17               ;examine and discard next bit
    brbc 0, zero_partial ; if 0 then no addition
```

```
    add r0, r16           ; it's 1 so addition required
```

zero\_partial:

```
    ror r0                ; shift result right, bringing in carry flag
    ror r1                ; shift right
    dec r18               ; decrement counter
    brne next_bit        ; if not zero then do the next bit
    ret                  ; come back home with result
```

```
; End of source code
```

## Dividing binary Numbers ([contents](#))

The AVR microcontrollers do not have hardware division, so here is a multiplying algorithm based on, among others, the following two sources:

1. [Textbook](#) "Some Assembly Required: Assembly Language Programming with the AVR Microcontroller" by Timothy S Margush (p439)
2. [Youtube](#) video "Restoring Division Algorithm for Unsigned Integer" by Mr. Arnab Chakraborty

```
; M A I N   P R O G R A M   I N I T
```

```
Main:
```

```
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer
    .equ fred = 240            ; fred = 0xF0
    .equ barney = 11           ; barney = 0x0B
    sei                        ; Enable interrupts
```

```
;   P R O G R A M   L O O P
```

```
Loop:
```

```
    clr r0                    ; where quotient will be
    clr r1                    ; where remainder will be
    ldi r16, fred             ; load dividend
    ldi r17, barney           ; load divisor
    mov r0, r16               ; move dividend to r0
    rcall divide              ; recall multiplying routine
    rjmp Loop
```

```
divide:
```

```
    ldi r19, 9                ; loop counter (1 byte plus 1)
```

```
divloop:
```

```
    rol r0                    ; roll left
    dec r19                   ; decrement counter
    breq exit                 ; if equal finish

    rol r1                    ; complete shift into dividend window
    sub r1, r17               ; trial division
    brbc 0, goesinto          ; branch if carry cleared

    add r1, r17               ; reverse subtraction
    bclr 0                    ; clear carry
    rjmp divloop
```

```
goesinto:
```

```
    bset 0                    ; set carry
    rjmp divloop
```

```
exit:
```

```
    ret
```

```
; End of source code
```



|                                                                                                                                                                     |                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SUB - Subtract without Carry (<a href="#">contents</a>)</b><br><br>Description:<br>Subtracts two registers and places the result in the destination register Rd. | <b>SBC - Subtract with Carry (<a href="#">contents</a>)</b><br><br>Description:<br>Subtracts two registers and subtracts with the C flag and places the result in the destination register Rd. |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
; *****
;   M A I N   P R O G R A M   I N I T
;   *****
```

Main:

```
    ldi rmp, Low(RAMEND)
    out SPL, rmp                ; Init LSB stack pointer
    .equ fred = 425             ; fred = 0x01A9
    .equ barney = 342           ; barney = 0x0156
    sei                        ; Enable interrupts
```

```
; *****
;   P R O G R A M   L O O P
;   *****
```

Loop:

```
    ldi r16, high(fred)         ; load high byte 0x01
    ldi r17, low(fred)          ; load low byte 0xA9
    ldi r18, high(barney)       ; load high byte 0x01
    ldi r19, low(barney)        ; load low byte 0x56

    sub r17, r19                ; subtract low byte
    sbc r16, r18                ; subtract high byte with carry

    rjmp Loop
```

```
; End of source code
```

**SUBI - Subtract without Carry ([contents](#))****Description:**

Subtracts a register and a constant and places the result in the destination register Rd. This instruction is working on Register R16 to R31 and is very well suited for operations on the X, Y and Z pointers.

**SBCI - Subtract with Carry ([contents](#))****Description:**

Subtracts a constant from a register and subtracts with the C flag and places the result in the destination register Rd.

```
; *****
;   M A I N   P R O G R A M   I N I T
;   *****
```

**Main:**

```
    ldi rmp, Low(RAMEND)
    out SPL, rmp                ; Init LSB stack pointer
    .equ fred = 425             ; fred = 0x01A9
    sei                        ; Enable interrupts
```

```
; *****
;   P R O G R A M   L O O P
;   *****
```

**Loop:**

```
    ldi r16, high(fred)         ; load high byte 0x01
    ldi r17, low(fred)          ; load low byte 0xA9

    subi r17, low(342)          ; subtract low byte
    sbci r16, high(342)         ; subtract high byte with carry

    rjmp Loop
```

**SLEEP - Sleep mode ([contents](#))****Description:**

This instruction sets the circuit in sleep mode defined by the MCU control register.

See “[Interval - making a real program](#)”

## SPM - Store Program Memory ([contents](#))

### Description:

SPM can be used to erase a page in the program memory, to write a page in the program memory (that is already erased), and to set boot loader lock bits. In some devices, the program memory can be written one word at a time, in other devices an entire page can be programmed simultaneously after first filling a temporary page buffer. In all cases, the program memory must be erased one page at a time. When erasing the program memory, the RAMPX, RAMPY, RAMPZ and Z register are used as page address. When writing the program memory, the RAMPZ and Z register are used as page or word address, and the R1:R0 register pair is used as data. R1 determines the instruction high byte, and R0 determines the instruction low byte... When setting the boot loader lock bits, the R1:R0 register pair is used as data. Refer to the device documentation for detailed description of SPM usage. This instruction can address the entire program memory.

R1 determines the instruction high byte, and R0 determines the instruction low byte.

-----OO-----

### The verdict

This “Jedi Master” command is about re-programming the bootloader of the AVR chip from within flash, for instance when you would like to update the “firmware”. This seems like an unlikely scenario for a beginner, so we’ve made a conscious decision not to look too closely at this until we have at least a “Ninja Master” understanding of the remainder of the commands, and some more experience programming in assembler.

It’s not unheard of to “brick” chip after chip learning how to use this command, and at the moment we see no application for which we would find it necessary or desirable to implement SPM in a slice of code.

From the [textbook](#) “Some Assembly Required: Assembly Language Programming with the AVR Microcontroller” by Timothy S Margush (p157) when considering the more useful [LPM](#):

*There is a complementary instruction, store program memory (SPM). However, this is not available to application programs. The SPM instruction can only be executed if it is fetched from the BLS of flash memory. This instruction can erase a section of flash and then copy bytes from SRAM into flash. To write into flash storage, an entire page (64 words) must be erased, and then new data written. This technique is used by the bootloader to write new programs into the application area. Each time you download your updated program to the processor, the bootloader is using this instruction to place the correct words in flash in preparation for running your application.*

If you do wish to dive down this particular rabbit hole, check [out this link](#) and [this video](#), both from Microchip, or [this page](#) from the “Engineers Garage” website.

## SWAP - Swap Nibbles ([contents](#))

### Description:

Swaps high and low nibbles in a register.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
; *****
```

#### Main:

```
    ldi rmp,Low(RAMEND)  
    out SPL,rmp                ; Init LSB stack pointer  
    .equ fred = 150            ; define fred 0x96, 0b10010110  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
; *****
```

#### Loop:

```
    clr r16                    ; zero the register  
    ldi r16, fred              ; load 0b10010110  
    swap r16                   ; swap nibbles, 0b01101001  
    nop  
    swap r16                   ; swap nibbles, 0b10010110  
    nop  
  
    rjmp loop
```

```
; End of source code
```

## TST - Test for Zero or Minus ([contents](#))

### Description:

Tests if a register is zero or negative. Performs a logical AND between a register and itself. The register will remain unchanged.

```
; *****  
;   M A I N   P R O G R A M   I N I T  
;   *****
```

Main:

```
    ldi rmp, Low(RAMEND)  
    out SPL, rmp                ; Init LSB stack pointer  
    .equ fred = 42              ; define fred (0x2A)  
    sei                        ; Enable interrupts
```

```
; *****  
;   P R O G R A M   L O O P  
;   *****
```

Loop:

```
    ldi r16, fred                ; set r16 to 42
```

countdown:

```
    subi r16, 5                  ; decrement by 5 each cycle  
    tst r16                      ; test r16 for minus  
    brbc 2, countdown           ; if positive countdown some more  
  
    rjmp Loop                    ; else start again
```

```
; End of source code
```

## WDR - Watchdog Reset ([contents](#))

### Description:

This instruction resets the Watchdog Timer. This instruction must be executed within a limited time given by the WD prescaler. See the Watchdog Timer hardware specification.

A variation on [Interval – making a real program](#), this version uses two watchdog timing intervals and therefore the instruction **wdr** to reset the timing. The flow of the program is as follows:

- define some constants such as pins and ports
- set up the watchdog interrupt jump
- code the interrupt routine (in this case empty)
- setup up ports and the watchdog timer register
- code the loop which sleeps either in one 1 second block (led on) or six 8 second blocks (led off)

```
;
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 / _____ | 8
; RESET o--|          |--o VCC
;      2 |          | 7
; PB3  o--|          |--o PB2
;      3 |          | 6
; PB4  o--|          |--o PB1
;      4 |          | 5
; GND  o--|          |--o PB0
;      | _____ |
;

.nolist
.include "tn13adef.inc"          ; Define device ATtiny13A
.list

; *****
;      H A R D W A R E
; *****

; *****
;      P O R T S   A N D   P I N S
; *****

.equ pOut = PORTB                ; Define a output port
.equ pDir = DDRB                 ; Define a direction port
.equ pIn = PINB                 ; Define an input port
.equ bMyPinO = PORTB0           ; Define an output pin
.equ bDir0 = DDB0               ; Define a direction pin
.equ bMyIn = PINB0              ; Define an input pin
```

```

; *****
;   A D J U S T A B L E   C O N S T
; *****

.equ clock=1200000                ; Define clock frequency

; *****
;   F I X   &   D E R I V.   C O N S T
; *****
;
.equ cTc0Clk = clock / 256        ; Define from clock

; *****
;           R E G I S T E R S
; *****

; free: R0 to R14
.def rSreg = R15                  ; Save/Restore status port
.def rmp = R16                   ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...

; *****
;           S R A M
; *****

.dseg
.org SRAM_START
sLabel1:
.byte 16 ; Reserve 16 bytes

; *****
;           C O D E
; *****

.cseg
.org 000000

; *****
; R E S E T   &   I N T - V E C T O R S
; *****
    rjmp Main                    ; Reset vector
    reti ; INT0
    reti ; PCI0
    reti ; OVF0
    reti ; ERDY
    reti ; ACI
    reti ; OC0A
    reti ; OC0B
    rjmp mytimer_ISR            ; watchdog timer interrupt
    reti ; ADCC

```

```

; *****
;   I N T - S E R V I C E   R O U T .
; *****

; Add all interrupt service routines

mytimer_ISR:

    ; as I understand it, cli and sei are automatic on
    ; entry and exit respectively of ISR

    in r16, SREG                ; save SREG
    push r16                   ; do this if you want to use

finishISR:

    pop r16                    ; do this if you pushed SREG above
    out SREG, r16              ; restore SREG

    reti

; *****
;   M A I N   P R O G R A M   I N I T
; *****

Main:
    ldi rmp, Low(RAMEND)
    out SPL, rmp                ; Init LSB stack pointer

    ldi r16, 0b00000001         ; output mode for PB0
    out DDRB, r16

    ; turn on PB0, enable internal pullups
    ; on the rest of the pins
    ldi r16, 0b11111111
    out PORTB, r16

; *****
;   P R O G R A M   L O O P
; *****

Loop:

    rcall waitabit              ; 1 second subroutine
    rcall waitalot             ; 6 x 8 second subroutine

    rjmp Loop

```



waitabit:

```
wdr                                ; reset watchdog timer

ldi r16, 0b01000110                ; 1 second timer
out WDTCR, r16

rcall sleepnow                      ; sleep subroutine

rcall flipbit                       ; flip the led, if off then on, etc.

ret
```

waitalot:

```
wdr                                ; reset watchdog timer

; 8 second timer
ldi r16, 0b01100001
out WDTCR, r16

; six lots of eight seconds into the countdown register
ldi r21, 6
```

countdown:

```
rcall sleepnow                      ; sleep subroutine
dec r21                             ; decrement counter
brbc 1, countdown                  ; if not zero then keep counting
rcall flipbit                       ; flip the led, if off then on, etc.
ret
```

sleepnow:

```
; sleep enable, power down mode
ldi r16, 0b00110000
out MCUCR, r16

; disable ADC
ldi r16, 0b00010000
out ADCSRA, r16

; disable interrupts to make sure the following
; timed sequence is not interrupted
cli

; disable BOD - must be done in a timed sequence
; as follows, write 1 to BODS and BODSE then within
; four clock cycles write 1 to BODS and 0 to BODSE
; then within three clock cycles enable sleep
; page 33 on the datasheet

ldi r16, 0b00000011
ldi r17, 0b00000010
out BODCR, r16
out BODCR, r17

sei
sleep

; sleep disable
ldi r16, 0b00000000
out MCUCR, r16

ret
```

flipbit:

```
in r16, PINB                ; what is the state of the pin?
ldi r17, 0b00000001        ; load a 1 for comparison
eor r16, r17               ; eor the two values
out PORTB, r16             ; bit is flipped and out

ret
```

; End of source code

## Some code which might be useful ([contents](#))

Here are some short programs that illustrate some useful programming tools in AVR assembler, as related to the Attiny13.

### PWM fade

In this code PB0 is faded in and out using pulse width modulation. Interestingly the LED appears to pulse fast then slow, but in fact it is linear. The problem is our human eyes which are too good at putting the pulses together to make a coherent light. This is a well known phenomena and can be “beaten” using a [logarithmic scale](#) instead of a linear scale.

```
;
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 /-----| 8
; RESET o--|      |--o VCC
;      2 |      | 7
; PB3  o--|      |--o PB2
;      3 |      | 6
; PB4  o--|      |--o PB1
;      4 |      | 5
; GND  o--|      |--o PB0
;      |-----|
;

.nolist
.include "tn13adef.inc"          ; Define device ATtiny13A
.list

; *****
;      H A R D W A R E
; *****

; *****
; P O R T S   A N D   P I N S
; *****

.equ pOut = PORTB                ; Define a output port
.equ pDir = DDRB                 ; Define a direction port
.equ pIn = PINB                  ; Define an input port
.equ bMyPinO = PORTB0            ; Define an output pin
.equ bDir0 = DDB0                ; Define a direction pin
.equ bMyIn = PINB0               ; Define an input pin
```

```

; *****
;   A D J U S T A B L E   C O N S T
; *****

.equ clock=1200000          ; Define clock frequency

; *****
;   F I X   &   D E R I V.   C O N S T
; *****
;
.equ cTc0Clk = clock / 256    ; Define from clock

; *****
;           R E G I S T E R S
; *****

; free: R0 to R14
.def rSreg = R15              ; Save/Restore status port
.def rmp = R16                ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...

; *****
;           S R A M
; *****

.dseg
.org SRAM_START
sLabel1:
.byte 16 ; Reserve 16 bytes

; *****
;           C O D E
; *****

.cseg
.org 000000

; *****
; R E S E T   &   I N T - V E C T O R S
; *****
    rjmp Main                ; Reset vector
    reti ; INT0
    reti ; PCI0
    reti ; OVF0
    reti ; ERDY
    reti ; ACI
    reti ; OC0A
    reti ; OC0B
    reti ; WDT
    reti ; ADCC

```

```
; *****
; I N T - S E R V I C E   R O U T .
; *****

; Add all interrupt service routines

; *****
; M A I N   P R O G R A M   I N I T
; *****
```

Main:

```
ldi rmp, Low(RAMEND)
out SPL, rmp                                ; Init LSB stack pointer

ldi r16, 0b00000001                        ; output mode for PB0
out DDRB, r16

.equ lowlight = 2                          ; low led set at 2 (from 255)
.equ highlight = 250                       ; high led set at 250 (from 255)

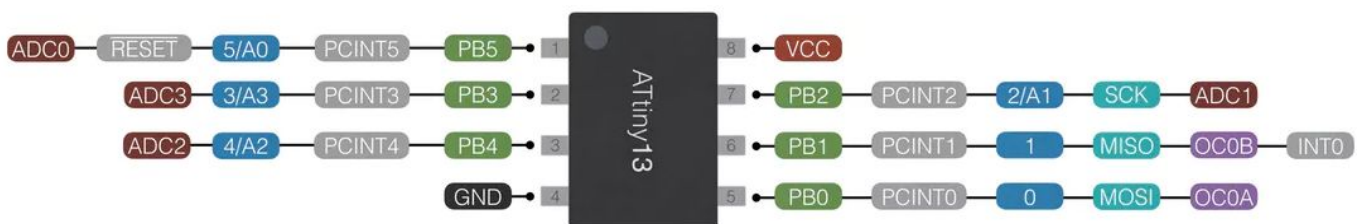
ldi r20, lowlight                          ; load low value
ldi r21, highlight                         ; load high value
mov r22, r20                              ; r22 will keep track of current value

; hardware pwm on channel A
; COM0A1(1), COM0A0(0), WGM01(1), WGM00(1)

ldi r16, 0b10000011
out TCCR0A, r16
```

| Bit           | 7      | 6      | 5      | 4      | 3 | 2 | 1     | 0     |        |
|---------------|--------|--------|--------|--------|---|---|-------|-------|--------|
| 0x2F          | COM0A1 | COM0A0 | COM0B1 | COM0B0 | - | - | WGM01 | WGM00 | TCCR0A |
| Read/Write    | R/W    | R/W    | R/W    | R/W    | R | R | R/W   | R/W   |        |
| Initial Value | 0      | 0      | 0      | 0      | 0 | 0 | 0     | 0     |        |

```
; channel A is output to PB0 according to the pinout diagram
```



; and so with fast PWM mode selected...

**Table 11-8.** Waveform Generation Mode Bit Description

| Mode | WGM02 | WGM01 | WGM00 | Timer/Counter Mode of Operation | TOP  | Update of OCRx at | TOV Flag Set on <sup>(1)(2)</sup> |
|------|-------|-------|-------|---------------------------------|------|-------------------|-----------------------------------|
| 0    | 0     | 0     | 0     | Normal                          | 0xFF | Immediate         | MAX                               |
| 1    | 0     | 0     | 1     | PWM (Phase Correct)             | 0xFF | TOP               | BOTTOM                            |
| 2    | 0     | 1     | 0     | CTC                             | OCRA | Immediate         | MAX                               |
| 3    | 0     | 1     | 1     | Fast PWM                        | 0xFF | TOP               | MAX                               |
| 4    | 1     | 0     | 0     | Reserved                        | –    | –                 | –                                 |
| 5    | 1     | 0     | 1     | PWM (Phase Correct)             | OCRA | TOP               | BOTTOM                            |
| 6    | 1     | 1     | 0     | Reserved                        | –    | –                 | –                                 |
| 7    | 1     | 1     | 1     | Fast PWM                        | OCRA | TOP               | TOP                               |

Notes: 1. MAX = 0xFF  
2. BOTTOM = 0x00

; ...the mode is set at clear OC0A on Compare Match, set OC0A at TOP

**Table 11-3.** Compare Output Mode, Fast PWM Mode<sup>(1)</sup>

| COM0A1 | COM0A0 | Description                                                                                      |
|--------|--------|--------------------------------------------------------------------------------------------------|
| 0      | 0      | Normal port operation, OC0A disconnected.                                                        |
| 0      | 1      | WGM02 = 0: Normal Port Operation, OC0A Disconnected.<br>WGM02 = 1: Toggle OC0A on Compare Match. |
| 1      | 0      | Clear OC0A on Compare Match, set OC0A at TOP                                                     |
| 1      | 1      | Set OC0A on Compare Match, clear OC0A at TOP                                                     |

```
; hardware pwm on channel B no prescaler
; CS00 (1)
ldi r16, 0b00000001
out TCCR0B, r16
```

| Bit           | 7     | 6     | 5 | 4 | 3     | 2    | 1    | 0    |        |
|---------------|-------|-------|---|---|-------|------|------|------|--------|
| 0x33          | FOC0A | FOC0B | – | – | WGM02 | CS02 | CS01 | CS00 | TCCR0B |
| Read/Write    | W     | W     | R | R | R/W   | R/W  | R/W  | R/W  |        |
| Initial Value | 0     | 0     | 0 | 0 | 0     | 0    | 0    | 0    |        |

**Table 11-9.** Clock Select Bit Description

| CS02 | CS01 | CS00 | Description                                             |
|------|------|------|---------------------------------------------------------|
| 0    | 0    | 0    | No clock source (Timer/Counter stopped)                 |
| 0    | 0    | 1    | clk <sub>I/O</sub> /(No prescaling)                     |
| 0    | 1    | 0    | clk <sub>I/O</sub> /8 (From prescaler)                  |
| 0    | 1    | 1    | clk <sub>I/O</sub> /64 (From prescaler)                 |
| 1    | 0    | 0    | clk <sub>I/O</sub> /256 (From prescaler)                |
| 1    | 0    | 1    | clk <sub>I/O</sub> /1024 (From prescaler)               |
| 1    | 1    | 0    | External clock source on T0 pin. Clock on falling edge. |
| 1    | 1    | 1    | External clock source on T0 pin. Clock on rising edge.  |

```
;
; *****
;   P R O G R A M   L O O P
; *****
```

Loop:

```
    rcall fadeup                ; call fadeup subroutine
    rcall fadtdown              ; call fadtdown subroutine

    rjmp Loop
```

fadeup:

```
    out OCR0A, r22              ; current value is output to OCR0A
    rcall wait5ms               ; pause 5 milliseconds
    subi r22, -1                ; subtract -1, which adds 1
    cp r22, r21                 ; have we reached our set limit?
    brbc 1, fadeup              ; no so continue to change value

    ret
```

fadtdown:

```
    out OCR0A, r22              ; current value is output to OCR0A
    rcall wait5ms               ; pause 5 milliseconds
    subi r22, 1                 ; subtract 1
    cp r22, r20                 ; have we reached our set limit?
    brbc 1, fadtdown            ; no so continue to change value

    ret
```

```
wait5ms:

; Generated by delay loop calculator
; at http://www.bretmulvey.com/avrdelay.html
;
; Delay 6 000 cycles
; 5ms at 1.2 MHz

    ldi    r18, 8
    ldi    r19, 202
L1:  dec    r19
     brne  L1
     dec    r18
     brne  L1
     nop

     ret

; End of source code
```





## Pseudo-random LED (8 bit lfsr) ([contents](#))

Blink on and off at “random” times. This program uses the timer overflow of the attiny13 in comparison mode. The code uses a very basic linear feedback shift register (LFSR) algorithm for generating pseudo-random numbers. The code relies on a non-zero seed (in this case decimal 41 or hexadecimal 0x29 (happy birthday) which is subject to a logical shift left. If carry is set in SREG, then the resultant byte is output to the timer compare (OCR0A) ready for the countdown. If carry is not set, then there is an exclusive or with the other value (0x8B) and then the OCR0A is set. These values can be changed as required as long as the seed is non-zero. Changing the xor value produces interesting sequences, and of course the sequences are always pseudo-random because they run exactly the same from reset. It is possible to seed it more randomly by using the ADC attached to an input pin and polling the voltage which will vary depending on the nature of what is attached to that pin (e.g. wire, schottky diode, etc). The following is the sequence produced by this pair of seed and xor value (red shading indicates when eor is used (brbs 0 (blue shading) after lsl) and green shading indicates the “random” number):

| Op    | Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 | Hex | Dec |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| start | 0     | 0     | 1     | 0     | 1     | 0     | 0     | 1     | 29  | 41  |
| lsl   | 0     | 1     | 0     | 1     | 0     | 0     | 1     | 0     | 52  | 82  |
| 8B    | 1     | 0     | 0     | 0     | 1     | 0     | 1     | 1     | 8B  | 139 |
| eor   | 1     | 1     | 0     | 1     | 1     | 0     | 0     | 1     | D9  | 217 |
| lsl   | 1     | 0     | 1     | 1     | 0     | 0     | 1     | 0     | B2  | 178 |
| lsl   | 0     | 1     | 1     | 0     | 0     | 1     | 0     | 0     | 64  | 100 |
| lsl   | 1     | 1     | 0     | 0     | 1     | 0     | 0     | 0     | C8  | 200 |
| 8B    | 1     | 0     | 0     | 0     | 1     | 0     | 1     | 1     | 8B  | 139 |
| eor   | 0     | 1     | 0     | 0     | 0     | 0     | 1     | 1     | 43  | 67  |
| lsl   | 1     | 0     | 0     | 0     | 0     | 1     | 1     | 0     | 86  | 134 |
| 8B    | 1     | 0     | 0     | 0     | 1     | 0     | 1     | 1     | 8B  | 139 |
| eor   | 0     | 0     | 0     | 0     | 1     | 1     | 0     | 1     | 0D  | 13  |
| lsl   | 0     | 0     | 0     | 1     | 1     | 0     | 1     | 0     | 1A  | 26  |
| 8B    | 1     | 0     | 0     | 0     | 1     | 0     | 1     | 1     | 8B  | 139 |
| eor   | 1     | 0     | 0     | 1     | 0     | 0     | 0     | 1     | 91  | 145 |

At only 1.2Mhz this is still too fast, so firstly the clock is divided by 1024 and then further a counter of 15 (0x0F) is used in register 19. As each cycle trips, the counter is decremented until zero, and then the counter reset, the LED flipped on or off depending on previous state, and the “random” number changed for the next cycle.

```

;
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 /-----| 8
; RESET o--|      |--o VCC
;      2 |      | 7
; PB3 o--|      |--o PB2
;      3 |      | 6
; PB4 o--|      |--o PB1
;      4 |      | 5
; GND o--|      |--o PB0
;      |-----|
;

.nolist
.include "tn13adef.inc"          ; Define device ATtiny13A
.list

; *****
;      H A R D W A R E
; *****

; *****
; P O R T S   A N D   P I N S
; *****

.equ pOut = PORTB                ; Define a output port
.equ pDir = DDRB                 ; Define a direction port
.equ pIn = PINB                  ; Define an input port
.equ bMyPinO = PORTB0            ; Define an output pin
.equ bDir0 = DDB0                ; Define a direction pin
.equ bMyIn = PINB0              ; Define an input pin

; *****
;      A D J U S T A B L E   C O N S T
; *****

.equ clock=1200000               ; Define clock frequency

; *****
; F I X   &   D E R I V .   C O N S T
; *****
;
.equ cTc0Clk = clock / 256       ; Define from clock

```

```

; *****
;           R E G I S T E R S
; *****

; free: R0 to R14
.def rSreg = R15           ; Save/Restore status port
.def rmp = R16             ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...

; *****
;           S R A M
; *****

.dseg
.org SRAM_START
sLabel1:
.byte 16 ; Reserve 16 bytes

; *****
;           C O D E
; *****

.cseg
.org 000000

;
; *****
; R E S E T   &   I N T - V E C T O R S
; *****
    rjmp Main                ; Reset vector
    reti ; INT0
    reti ; PCIO
    reti ; OVFO
    reti ; ERDY
    reti ; ACI
    rjmp COMISR              ; compare interrupt triggers COMISR routine
    reti ; OC0B
    reti ; WDT
    reti ; ADCC

```

```

;
; *****
; I N T - S E R V I C E   R O U T .
; *****
;
; Add all interrupt service routines

COMISR:
    push r16                ; save global registers on stack
    push r17
    push r18
    in r16, SREG            ; save SREG
    push r16                ; do this if you want to use
                            ; r16 in your interrupt routine
    dec r19                 ; decrement counter
    brbc 1, finishisr       ; if not zero, countdown again

rollarandom:
    ldi r19, 15             ; reset counter
    lsl r20                 ; logical shift left
    brbs 0, carried         ; branch if carry set, else xor
    eor r20, r21            ; xor with 0x8B

carried:

    out OCR0A, r20          ; set comparison value

    in r16, PINB            ; read status of the pins
    ldi r17, 0b00000001     ; load comparison
    eor r16, r17            ; compare and flip PB0
    out PORTB, r16          ; and make it so

finishisr:

    pop r16                 ; do this if you pushed SREG above
    out SREG, r16           ; restore SREG
    pop r18                 ; restore global registers
    pop r17
    pop r16
    reti

```

```

;
; *****
;   M A I N   P R O G R A M   I N I T
; *****
;

Main:
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer
    ldi r16,0b00000010        ; set to CTC mode
    out TCCR0A, r16            ; and make it so
    ldi r20, 0x29              ; load random seed
    ldi r21, 0x8B              ; load xor value
    out OCR0A, r20              ; first compare value
    ldi r16,0b00000101        ; set timer prescaler to /1024
    out TCCR0B, r16            ; and make it so
    ldi r16, 0b00000100        ; set timer to compare
    out TIMSK0, r16            ; and make it so

    ldi r16, 0b00000001        ; output mode for PB0
    out DDRB, r16              ; and make it so

    ; turn on PB0, enable internal pullups
    ; on the rest of the pins
    ldi r16, 0b11111111
    out PORTB, r16              ; and make it so
    ldi r19, 15                 ; initialise counter to 15 (0x0F)

    sei ; Enable interrupts

;
; *****
;   P R O G R A M   L O O P
; *****
;

Loop:

rjmp Loop

; End of source code

```

## Button Debouncing, input and output ([contents](#))

Nothing seems to generate more aggravation than the gentle art of button debouncing. All sorts of schemes are employed to make sure that the contacts on a button don't provide a set of false readings and ruin your day. Having looked at all of the options, we settled on a timer approach. A timer counts up and at the trigger or overflow point, the state of the button is checked. Here is the plan:

1. Set up one output pin (PB0) and one input pin (PB4)
2. Let a counter cycle to provide a suitable gap between checking the button, in this case the calculation is  $2 \times 256 \times 1 / 1200000$  or about  $400\mu\text{s}$
3. Allow a few ms for everything to settle down before the start of the first cycle
4. Check the button and the state of the LED each 2 cycles (00, 01, 00, 01, etc.,)

The code part was relatively easy, but the electronic part needed two things to work properly - firstly all pins not in use had to have internal pullups (no floaters), and secondly a little settle time (5ms) was needed after setup to make the little microcontroller happy.

```
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 /-----| 8
; RESET o--|      |--o VCC
;      2 |      | 7
; PB3  o--|      |--o PB2
;      3 |      | 6
; PB4  o--|      |--o PB1
;      4 |      | 5
; GND  o--|      |--o PB0
;      |-----|
;
;
; *****
; *                                     *
; *                                     *
; *****
;
.nolist
.include "tn13adef.inc"      ; Define device ATtiny13A
.list

; *****
;      H A R D W A R E
; *****
; P O R T S   A N D   P I N S
; *****

.equ pOut = PORTB           ; Define a output port
.equ pDir = DDRB            ; Define a direction port
.equ pIn = PINB             ; Define an input port
.equ bMyPinO = PORTB0       ; Define an output pin
.equ bDir0 = DDB0           ; Define a direction pin
.equ bMyIn = PINB0          ; Define an input pin
```

```

;
; *****
;   A D J U S T A B L E   C O N S T
; *****
;
.equ clock=1200000                ; Define clock frequency
;
; *****
;   F I X   &   D E R I V.   C O N S T
; *****
;
.equ cTc0Clk = clock / 256        ; Define from clock
;
; *****
;           R E G I S T E R S
; *****
;
; free: R0 to R14
.def rSreg = R15                  ; Save/Restore status port
.def rmp = R16                    ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...
;
; *****
;           S R A M
; *****
;
.dseg
.org SRAM_START
sLabel1:
.byte 16                          ; Reserve 16 bytes
;
; *****
;           C O D E
; *****
;
.cseg
.org 000000
;
; *****
; R E S E T   &   I N T - V E C T O R S
; *****
rjmp Main                          ; Reset vector
reti ; INT0
reti ; PCIO
rjmp changeflag                    ; OVFO
reti ; ERDY
reti ; ACI
reti ; OC0A
reti ; OC0B
reti ; WDT
reti ; ADCC

```

```

;
; *****
;   I N T - S E R V I C E   R O U T .
; *****
;
; Add all interrupt service routines
;

```

change flag:

```

    push r16                ; save global registers on stack
    push r17
    push r18
    in r16, SREG            ; save SREG
    push r16                ; do this if you want to use
                           ; r16 in your interrupt routine

    ldi r17, 0b00000001    ; load a 1 for checking
    eor r20, r17           ; and flip the flag in r20

```

finish isr:

```

    pop r16                ; do this if you pushed SREG above
    out SREG, r16          ; restore SREG
    pop r18                ; restore global registers
    pop r17
    pop r16

    reti

```



```

; *****
;   M A I N   P R O G R A M   I N I T
; *****
;
Main:
    ldi rmp,Low(RAMEND)
    out SPL,rmp                ; Init LSB stack pointer

    ldi r16, 0b00000001        ; load 0x01 into r16
    out DDRB, r16              ; all inputs except PB0
    out TCCR0B, r16            ; clock divider 1

    ldi r16, 0b11101110        ; internal pullups except on PB0, PB4
    out PORTB, r16             ; and out

    ldi r16, 0b00000000        ; normal clock function
    out TCCR0A, r16            ; and out to the clock

    ldi r16, 0b00000010        ; TOIE0: Timer/Counter0
    out TIMSK0, r16           ; Overflow Interrupt Enable

    ; these are our four possible states ready for checking later
    ; PB0(0), PB4(0) : button off, LED off
    ; PB0(1), PB4(0) : button off, LED on
    ; PB0(0), PB4(1) : button on, LED off
    ; PB0(1), PB4(1) : button on, LED on

    ldi r16, 0b00000000
    mov r0, r16

    ldi r16, 0b00000001
    mov r1, r16

    ldi r16, 0b00010000
    mov r2, r16

    ldi r16, 0b00010001
    mov r3, r16

    ldi r20, 0b00000000        ; the toggle switch for checking button

    ; a seemingly necessary 5ms break to settle everything

    rcall settletime

    sei                        ; Enable interrupts

```

```

;
; *****
;   P R O G R A M   L O O P
; *****
;
Loop:
    ldi r17, 0b00000001        ; load a 0x01 to check
    eor r17, r20               ; toggle the r20 flag
    brbc 1, finishloop         ; if r20 is not zero, loop again

    rcall themagic             ; but if r20 = 1, check the button

finishloop:

    rjmp Loop

settletime:
; Generated by delay loop calculator
; at http://www.bretmulvey.com/avrdelay.html
;
; Delay 6 000 cycles
; 5ms at 1.2 MHz

    ldi r18, 8
    ldi r19, 202
L1: dec r19
    brne L1
    dec r18
    brne L1
    nop

themagic:

; For x,y if x is button and y is light then we want
; 0,0 -> button off, LED off so LED stays off (0x00 in r0)
; 0,1 -> button off, LED on so LED turns off (0x01 in r1)
; 1,0 -> button on, LED off so LED turns on (0x10 in r2)
; 1,1 -> button on, LED on so LED stays on (0x11 in r3)

checkbutton:

    in r16, PINB                ; state of the pins
    andi r16, 0b00010000        ; mask out all but PB4
    in r17, PINB                ; state of the pins
    andi r17, 0b00000001        ; mask out all but PB0
    or r17, r16                 ; "or" the values ready for comparison

```

```

mov r16, r17          ; move "or" result in readiness for testing
eor r16, r0           ; is LED off and button off?
brbs 1, nochange      ; yes so no change

mov r16, r17          ; move "or" result in readiness for testing
eor r16, r1           ; is LED on and button off?
brbs 1, flipit       ; yes so go flip the LED switch

mov r16, r17          ; move "or" result in readiness for testing
eor r16, r2           ; is LED off and button on?
brbs 1, flipit       ; yes so go flip the LED switch

mov r16, r17          ; move "or" result in readiness for testing
eor r16, r3           ; is LED on and button on?
brbs 1, nochange      ; yes so no change

rjmp nochange         ; catches the non-existent fifth case

```

flipit:

```

in r16, PINB          ; state of the pins
eor r16, r1           ; flip the bit
out PORTB, r16        ; and out to the PORTS

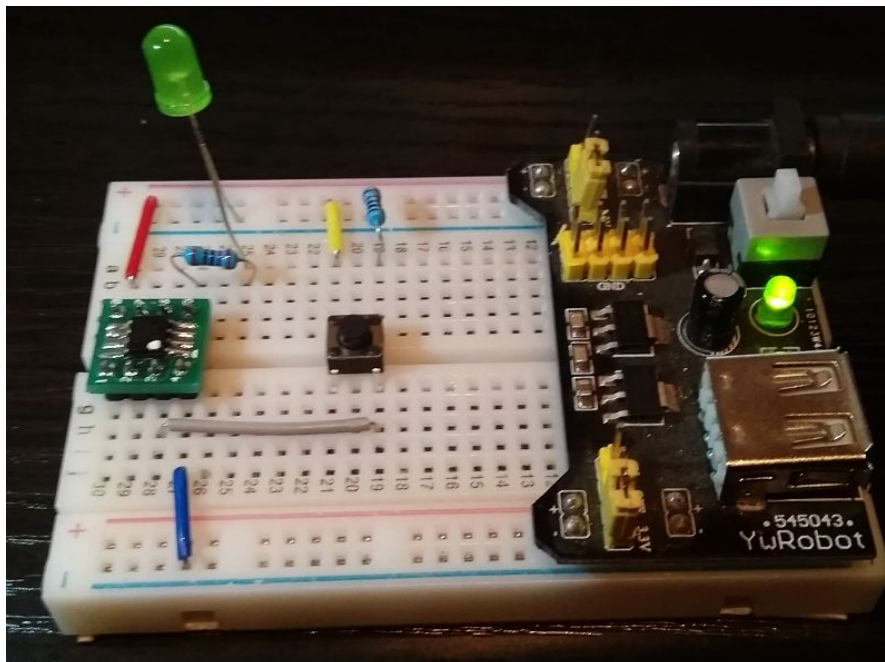
```

nochange:

```
ret
```

; End of source code

And here is a picture of the setup in "action"



## Reading the ADC and comparing the result with a constant ([contents](#))

The real world is analog but we need digital for our microprocessors and when making decisions. The Analog to Digital Converter (ADC) has a great deal of flexibility to allow us to do exactly that. In this application we will have an analog input provided by a potentiometer, and a digital output in the guise of a simple LED. Here is the plan:

1. Set up one output pin (PB0) and one input pin (PB4)
2. Let a counter cycle to provide a suitable gap between checking the analog input, in this case the calculation is  $2 \times 256 \times 1 / 1200000$  or about  $400\mu\text{s}$
3. Allow a few ms for everything to settle down before the start of the first cycle
4. Check the analog input and change the LED state as appropriate.

Capacitors feature in the final circuit to prevent any noise from ruining our day.

```
; Device: ATtiny13A, Package: 8-pin-PDIP_SOIC
;
;
;      1 /-----| 8
; RESET o--|      |--o VCC
;      2 |      | 7
; PB3  o--|      |--o PB2
;      3 |      | 6
; PB4  o--|      |--o PB1
;      4 |      | 5
; GND  o--|      |--o PB0
;      |-----|
;
;
; *****
; *                                     *
; *                                     *
; *****

.nolist
.include "tn13adef.inc"      ; Define device ATtiny13A
.list

; *****
;      H A R D W A R E
; *****
;
; *****
; P O R T S   A N D   P I N S
; *****

.equ pOut = PORTB             ; Define a output port
.equ pDir = DDRB              ; Define a direction port
.equ pIn = PINB               ; Define an input port
.equ bMyPinO = PORTB0         ; Define an output pin
.equ bDir0 = DDB0             ; Define a direction pin
.equ bMyIn = PINB0            ; Define an input pin
```

```

;
; *****
;   A D J U S T A B L E   C O N S T
; *****
;
.equ clock=1200000          ; Define clock frequency
;
; *****
;   F I X   &   D E R I V.   C O N S T
; *****
;
.equ cTc0Clk = clock / 256    ; Define from clock
;
; *****
;           R E G I S T E R S
; *****
;
; free: R0 to R14
.def rSreg = R15              ; Save/Restore status port
.def rmp = R16                ; Define multipurpose register
; free: R17 to R29
; used: R31:R30 = Z for ...
;
; *****
;           S R A M
; *****
;
.dseg
.org SRAM_START
sLabel1:
.byte 16                      ; Reserve 16 bytes
;
; *****
;           C O D E
; *****
;
.cseg
.org 000000
;
; *****
; R E S E T   &   I N T - V E C T O R S
; *****
rjmp Main                    ; Reset vector
reti ; INT0
reti ; PCIO
rjmp changeflag              ; OVFO
reti ; ERDY
reti ; ACI
reti ; OC0A
reti ; OC0B
reti ; WDT
reti ; ADCC

```

```
; *****
; I N T - S E R V I C E   R O U T .
; *****
;
; Add all interrupt service routines
```

changeflag:

```
    push r16                ; save global registers on stack
    push r17
    push r18
    in r16, SREG            ; save SREG
    push r16                ; do this if you want to use
                           ; r16 in your interrupt routine

    ldi r17, 0b00000001    ; a single bit for comparison
    eor r20, r17           ; flip the switch
```

finishisr:

```
    pop r16                 ; do this if you pushed SREG above
    out SREG, r16           ; restore SREG
    pop r18                 ; restore global registers
    pop r17
    pop r16
    reti
```

```
; *****
; M A I N   P R O G R A M   I N I T
; *****
```

Main:

```
    .equ threshold = 400    ; threshold value (0-1023)

    ldi rmp, Low(RAMEND)
    out SPL, rmp            ; Init LSB stack pointer

    ldi r16, 0b00000001    ; PB0 as an output
    out DDRB, r16          ; and make it so

    ldi r16, 0b11111110    ; pullups on all pins but PB0
    out PORTB, r16         ; and out to PORTB

    ldi r16, 0b00010000    ; turn off digital input on PB4 (datasheet p81)
    out DIDR0, r16         ; and out to DIDR0

    ldi r16, 0b00000000    ; load 0x00 to r16
    out TCCR0A, r16        ; Normal port operation
    out ADCSRB, r16        ; free running mode

    ldi r16, 0b00000011    ; pre-scaler is clock divided by 64
    out TCCR0B, r16        ; and out to TCCR0B
```

```

    ldi r16, 0b00000010      ; Timer/Counter0 Overflow Interrupt Enable
    out TIMSK0, r16          ; and ready to overflow

    ldi r16, 0b00000010      ; select ADC2 (PB4) (datasheet page 93)
    out ADMUX, r16           ; and out

    ldi r16, 0b10000011      ; ADC enable, division factor 8
    out ADCSRA, r16          ; and out

    ldi r20, 0b00000000      ; initially flag is zero

    sbi ADCSRA, 6             ; an initialising conversion to start with

    rcall settletime          ; just stay calm for a few ms

    sei                       ; Enable interrupts

; *****
;   P R O G R A M   L O O P
; *****

Loop:

    ldi r17, 0b00000001      ; a comparison bit
    eor r17, r20             ; and test the flag
    brbc 1, finishloop       ; if not 1, keep looping

    rcall themagic           ; else let's get an ADC reading

finishloop:

    rjmp Loop

settletime:
; Generated by delay loop calculator
; at http://www.bretmulvey.com/avrdelay.html
; Delay 6 000 cycles
; 5ms at 1.2 MHz

    ldi r18, 8
    ldi r19, 202
L1: dec r19
    brne L1
    dec r18
    brne L1
    nop

themagic:

    sbi ADCSRA, 6            ; start a conversion

```

completetime:

```
in r16, ADCSRA          ; load ADCSRA register
andi r16, 0b01000000    ; mask all but bit 6
ldi r17, 0b01000000     ; ready to test the bit
eor r16, r17             ; is it zero yet?
brbs 1, completetime     ; nope so go back and wait some more
```

```
in r16, ADCL             ; conversion complete so load low byte
in r17, ADCH             ; and then high byte (must be in this
order)
```

```
ldi r18, low(threshold)  ; load low part of threshold
ldi r19, high(threshold) ; and then high part of threshold
```

```
cp r16, r18              ; compare low byte
cpc r17, r19             ; compare with carry high byte
brbc 2, ledon            ; if higher then flick on LED
```

ledoff:

```
cbi PORTB, 0            ; clear bit 0 of PORTB
rjmp endmagic           ; and finish up
```

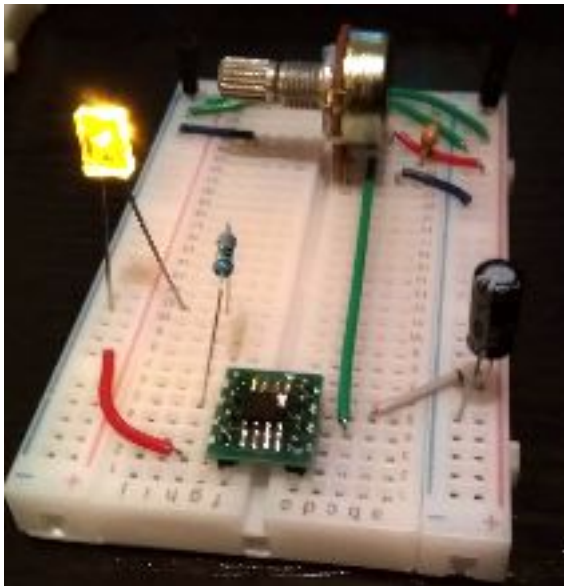
ledon:

```
sbi PORTB, 0            ; set bit 0 of PORTB
```

endmagic:

```
ldi r20, 0x00           ; clear flag for another cycle
ret
```

; End of source code



***That's it for now, but if you have enjoyed the examples and explanations, please consider a small donation to keep the house warm and feed the animals!***

***Your feedback and suggestions would also be really welcome - please email me at [peckmaths@gmail.com](mailto:peckmaths@gmail.com)***

**Donate**



